Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «УРАЛЬСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ ИМЕНИ
ПЕРВОГО ПРЕЗИДЕНТА РОСИИ Б.Н.ЕЛЬЦИНА»
(ИРИТ-РТФ
ЦЕНТР УСКОРЕННОГО ОБУЧЕНИЯ)

ОТЧЕТ
ПО ПРОЕКТНОМУ ПРАКТИКУМУ
АВТОМАТИЗАЦИЯ И РАЗРАБОТКА ПО

Куратор _____ К.А. Аксенов
подпись, дата

Екатеринбург 2026

СПИСОК ИСПОЛНИТЕЛЕЙ

Студент группы
РИЗ-240916у _____ А.В. Масляков
подпись, дата

Студент группы
РИЗ-240916у _____ А.В. Казанцев
подпись, дата

Студент группы
РИЗ-240916у _____ К.Н. Маркин
подпись, дата

Студент группы
РИЗ-240916у _____ Н.В. Палинов
подпись, дата

Студент группы
РИЗ-240916у _____ К.Р. Мансуров
подпись, дата

Студент группы
РИЗ-240916у _____ Е.М. Разорин
подпись, дата

Студент группы
РИЗ-240916у _____ Д.С. Скобелин
подпись, дата

СОДЕРЖАНИЕ

Введение . . . 5

1 Описание предметной области . . . 6

1.1 Ананлиз специфики предметной области . . . 6

1.2 Основные понятия и термины . . . 7

1.3 Обзор близких технических решений . . . 8

2 База знаний . . . 10

2.1 Структура фактов и их семантика . . . 10

2.2 Принципы построения продукционных правил . . . 11

2.3 Формирование правил . . . 12

2.4 Примеры правил (фрагмент базы) . . . 13

3 Проверка непротиворечивости . . . 15

3.1 Методы проверки . . . 15

3.2 Результаты анализа . . . 16

3.3 Исправление противоречий . . . 17

3.3.1 Локальная переработка трех конфликтных пар . . . 18

3.3.2 Механизм разрешения конфликтов . . . 19

3.3.3 Дополнительные меры стабилизации базы знаний . . . 19

4 Средства разработки . . . 21

4.1 Обоснование выбора языка программирования . . . 21

4.2 Описание используемых библиотек . . . 22

4.3 Инструменты разработки . . . 23

5 Техническое решение . . . 24

5.1 Архитектура системы . . . 24

5.2 Схема работы . . . 24

5.3 Алгоритм реализации . . . 25

6 Тестирование прототипа . . . 27

Заключение 28

Список использованных источников 29

Приложение А Исходный код прототипа экспертной системы 30

ВВЕДЕНИЕ

Туризм остается сложной сферой, где решение клиента зависит сразу от нескольких факторов: цели поездки, условий отдыха, бюджета и состава группы. В такой ситуации человеку трудно быстро выбрать подходящий вариант без структурированной консультации [1].

Для туристских услуг особенно важна точная привязка предложения к потребностям пользователя. Если такая привязка слабая, рекомендация получается общей и теряет практическую ценность [2].

Цифровые инструменты в туризме развиваются, и вместе с этим растет запрос на понятные и прозрачные механизмы принятия решений [3]. Для прикладных систем это означает, что важно не только выдать ответ, но и показать логику, по которой он получен [2].

В данной работе эта задача решается через прототип продукционной экспертной системы «Консультант по путешествиям». Выбран подход с явными правилами вида «ЕСЛИ -> ТО», потому что он упрощает проверку рекомендаций и позволяет объяснять результат пользователю [4].

Практическая значимость проекта состоит в объединении трех функций: структурированный ввод фактов, автоматический выбор рекомендации и explain-режим с прямым и обратным выводом. Такой формат подходит для учебной разработки и для поэтапного расширения базы знаний [3].

Таким образом, тема экспертного консультирования в туризме является актуальной с точки зрения качества сервиса и цифровизации отрасли. Далее в отчете последовательно рассматриваются предметная область, понятийная база и особенности решений, на которых строится прототип [5].

1 Описание предметной области

1.1 Ананлиз специфики предметной области

Разрабатываемое приложение относится к сфере туристско-рекреационных услуг и решает задачу консультирования пользователя по выбору формата поездки. Предметная область туризма рассматривается как социально-экономическая система, в которой взаимодействуют субъект (турист с его мотивами и ограничениями) и объект (туристский регион, предприятия и организации, формирующие услугу) [1,5].

Для экспертной системы это означает, что рекомендация не может строиться только по одному признаку. Решение формируется на пересечении цели поездки, условий поездки и профиля туриста. В материалах по основам и рекреационному туризму подчеркивается, что качество услуги определяется прежде всего достижением основной цели путешествия, а не только уровнем отдельных сервисов [4,5]. Поэтому в базе знаний критична связка «мотив поездки -> тип туристской программы».

С прикладной точки зрения на логику правил влияют четыре группы характеристик предметной области. Первая группа: функции туризма (экономическая, социальная, гуманитарная и воспитательная), которые задают возможные пользовательские запросы от восстановления ресурсов до познавательных и деловых целей [1,2]. Вторая группа: внешние факторы (природно-климатические условия, экономическая ситуация, политико-правовые ограничения, технологическая доступность, экологическая обстановка). Третья группа: внутренние факторы отрасли (уровень инфраструктуры, формат обслуживания, степень дифференциации предложений). Четвертая группа: сезонность, из-за которой меняются доступность и релевантность рекомендаций.

Для прототипа «Консультант по путешествиям» это задает требования к структуре фактов и правил. Факты должны отражать: цель поездки, предпочтительный формат активности, ограничения по условиям пребывания, ожидаемый уровень сервиса и безопасности. Правила должны классифицировать запрос в один или несколько классов программного туризма (оздоровительный, познавательный, деловой, экологический, спортивный, обучающий, семейный), после чего выдавать целевую рекомендацию по типу тура и формату обслуживания [4,5].

Отдельно значима компонента безопасности: при проектировании туристских услуг учитываются риски среды, физические нагрузки, биологические угрозы и личная безопасность. Для экспертной системы это выражается в необходимости включать в консультацию хотя бы базовые предупреждения и условия применимости рекомендации к профилю туриста [1,3].

1.2 Основные понятия и термины

- туризм: путешествие и временное пребывание вне обычной среды, не связанное с оплачиваемой деятельностью в месте пребывания;
- турист: ночующий посетитель, совершающий поездку вне обычной среды в установленных временных пределах;
- посетитель: общее понятие для лиц, путешествующих вне обычной среды; включает туристов и экскурсантов;
- экскурсант: однодневный посетитель без ночевки в месте посещения;
- внутренний туризм: поездки в пределах собственной страны вне обычной среды;
- международный туризм: поездки за пределы страны постоянного проживания вне обычной среды;
- программное обслуживание: целевой комплекс услуг, объединенных мотивом поездки и ориентированных на конкретного потребителя;
- программа обслуживания: структурированный по времени набор услуг, обеспечивающий достижение цели путешествия;
- рекреационная деятельность: деятельность, направленная на восстановление физических и психоэмоциональных ресурсов человека;
- мотивация в туризме: совокупность причин, определяющих выбор направления, формата отдыха и туристской фирмы;
- сезонность: регулярные колебания туристского спроса с выраженными периодами пиков и спадов;

– туристские ресурсы: природные, культурно-исторические и социокультурные объекты, формирующие привлекательность территории;
– туристская инфраструктура: средства размещения, питания, транспорта, досуга и информационного сопровождения, обеспечивающие реализацию турпродукта;
– безопасность в туризме: учет и снижение рисков, связанных с условиями среды, нагрузками, биологическими и социальными угрозами [5].

1.3 Обзор близких технических решений

Сопоставление выполнено по материалам анализа главных страниц `ostrovok`, `turscanner` и `tripster` относительно текущего прототипа «Консультант по путешествиям». Критерии: полнота структурированного ввода, пригодность фактов к продукционным правилам и возможность объяснимого вывода [1,2,4].

Текущая система в проекте ориентирована на консультацию по правилам: пользователь задает формализованные параметры поездки, а модуль правил формирует рекомендацию. Поэтому сравнение аналогов проводилось с позиции применимости к экспертной диагностике, а не только к каталожному поиску. Сравнительные характеристики приведены в 1.

Таблица 1 — Название

Решение	**Сильные стороны**	**Узкие места**	**Сравнение с текущей системой**
Ostrovok	Направление, даты, состав гостей и тип поездки заданы явно.	Слабо выражены мотив поездки, бюджетные ограничения и профиль группы для классификации типа тура.	Близок по структуре ввода, но в меньшей степени ориентирован на экспертный вывод.
Turscanner	Явные фильтры сезонности, города вылета и признака «без визы».	Короткая анкета: нет цели, длительности, состава группы и предпочтений формата отдыха.	Лучше отражает внешние ограничения, но хуже поддерживает персонализированные правила.
Tripster	Быстрый поиск направления с подсказками.	На главной форме почти нет структурированных туристских фактов кроме направления.	Эффективен как навигация по каталогу, но заметно слабее текущего прототипа для экспертной диагностики.

Текущий прототип методически ближе к целевой архитектуре экспертной системы. Из аналогов рационально заимствовать компактность формы (`Ostrovok`) и явные сезонно-визовые фильтры (`Turscanner`) [2].

2 База знаний

База знаний прототипа экспертной системы «Консультант по путешествиям» сформирована в продукционной модели вида «ЕСЛИ (условия), ТО (рекомендация)» и реализована как совокупность формализованных фактов и правил вывода. Такой формат выбран как наиболее прозрачный для учебного прототипа: каждое решение системы может быть разложено на проверяемые предпосылки, а сами предпосылки выражены в ограниченном наборе пользовательских признаков [2,5].

При проектировании базы знаний использовался принцип предметной достаточности: в модель включались не все возможные характеристики туриста, а только те, которые непосредственно влияют на тип программы обслуживания и могут быть надежно получены в короткой анкете. Этот подход согласуется с положением о целевой природе туристской услуги, где ключевым критерием качества является достижение основной цели путешествия [4]. Дополнительно учитывалось, что туризм как социально-экономическая система зависит от внешних (в том числе природно-климатических и экономических) и внутренних факторов отрасли [1,2].

2.1 Структура фактов и их семантика

Фактологическая часть базы знаний организована как единый реестр признаков, из которого одновременно формируются поля ввода и факты для движка правил. В реализации это обеспечивается schema-driven-подходом: факты описываются в едином наборе спецификаций и автоматически используются формой и модулем вывода. Такая организация минимизирует расхождения между интерфейсом и логикой вывода, поскольку изменения в составе признаков выполняются в одном месте.

Для рассматриваемой версии системы ядро составляют пять типов фактов, применяемых в продукционных правилах:

- `climate` (климат поездки): теплый, умеренный, холодный;
- `travel_type` (предпочтительный формат отдыха): спокойный или активный;
- `budget` (уровень бюджета): низкий, средний, высокий;

- `trip_days` (длительность поездки): короткая, средняя, длинная (в ряде правил через пороги по числу дней);
- `companions` (состав путешествующих): один, пара, семья, друзья.

Выбор именно этих фактов обусловлен их высокой объяснительной способностью. Факт `travel_type` отражает мотив и ожидаемую активность туриста, что напрямую связано с принципом «цель поездки -> тип программы» [4,5]. Факты `climate` и `trip_days` описывают условия реализации программы (сезонность, нагрузка, доступность активностей), а `budget` и `companions` уточняют ограничения и формат сервиса, что соответствует социально-экономической природе туристского спроса [1,2].

Отдельное методическое решение касается дискретизации: вместо непрерывных и трудно верифицируемых оценок применяются ограниченные категории. Например, бюджет и длительность интерпретируются не как произвольные значения, а как интервалы и пороги («до 3 дней», «более 10 дней», «более 14 дней»). Это упрощает построение условий, снижает риск неоднозначной интерпретации и делает вывод воспроизводимым при повторном вводе тех же данных [2].

2.2 Принципы построения продукционных правил

Конструирование правил выполнялось не как механическое перечисление комбинаций, а по набору проектных принципов.

Первый принцип: целевая релевантность рекомендации. Каждое правило должно вести к типу тура, который логически следует из доминирующей цели пользователя. Например, комбинация «теплый климат + спокойный отдых» приводит к пляжному сценарию, тогда как «холодный климат + активный отдых» ориентирует на горнолыжный формат [5].

Второй принцип: модульность условий. Условия группируются по функциональным блокам (среда поездки, поведенческий профиль, ресурсные ограничения, состав группы). Это позволяет добавлять новые правила без перестройки существующих, а также облегчает перенос знаний в код движка.

Третий принцип: градация по уточнению. Базовые правила задают общий класс рекомендации по 1-2 признакам, а уточняющие правила вводят дополнительную конкретизацию по бюджету, длительности и составу компании. Такая стратификация нужна для контролируемого роста базы знаний: число правил увеличивается за счет осмысленного уточнения, а не за счет случайных комбинаций [2].

Четвертый принцип: разрешение конфликтов по специфичности. Потенциально пользовательский профиль может удовлетворять нескольким правилам. Поэтому более специфичные правила (с большим числом условий или с более «узкими» ограничениями) интерпретируются как приоритетные по сравнению с общими. В реализации этот подход поддерживается механизмом приоритетов (`priority`) и `salience`.

Пятый принцип: полнота покрытия и наличие правила по умолчанию. Для сохранения устойчивости консультации предусмотрено завершающее правило «универсальный тур», применяемое, если ни одна специализированная продукция не сработала. Это исключает ситуацию пустого результата и позволяет корректно завершать консультацию при редких или нетипичных сочетаниях фактов.

Шестой принцип: объяснимость. Каждое правило сопоставлено с формализованным набором условий, что позволяет в explain-режиме получить цепочку срабатывания (matched rules, selected rule, шаги проверок). Для учебной экспертной системы этот аспект является принципиальным: пользователь получает не только итог, но и основания выбора.

2.3 Формирование правил

Формирование реализовано через последовательное расширение базовой матрицы решений. Исходный слой строился на сочетании `climate` и `travel_type`, поскольку именно эта пара дает первичную классификацию сценария поездки (пляжный, экскурсионный, зимний активный и т.д.) [5]. Далее матрица расширялась тремя направлениями:

1. Экономическое уточнение по `budget`: выделение бюджетных, стандартных и премиальных вариантов [2].
2. Временное уточнение по `trip_days`: различение мини-поездок, стандартных и длинных туров [5].

3. Социальное уточнение по `companions`: адаптация рекомендации под одиночных туристов, пары, семьи и группы друзей [3].

Таким образом, база знаний сформирована как комбинация общего и частного уровней. На общем уровне правила покрывают главные траектории выбора, на частном уровне они повышают персонализацию рекомендации. При этом избегались два типовых дефекта продукционных систем: дублирование семантически одинаковых правил и прямые противоречия между рекомендациями в одинаковых условиях.

В практическом смысле использована следующая процедура качества: (а) проверка формальной корректности условий, (б) проверка доменной обоснованности рекомендации, (в) проверка включенности правила в общую структуру приоритетов, (г) тестирование сценария «ввод -> вывод -> сохранение объяснения». Такой порядок обеспечивает согласованность базы знаний с архитектурой приложения и упрощает дальнейшее масштабирование [3].

2.4 Примеры правил (фрагмент базы)

Полный перечень правил вынесен в рабочую документацию проекта; в отчете приведены только репрезентативные примеры.

1. Если климат теплый и отдых спокойный, то рекомендовать пляжный отдых.
2. Если климат холодный и отдых активный, то рекомендовать горнолыжный курорт.
3. Если бюджет высокий и длительность больше 10 дней, то рекомендовать дальние страны.
4. Если компания семья и климат умеренный, то рекомендовать семейный отдых.
5. Если отдых активный и длительность больше 7 дней, то рекомендовать приключение.
6. Если бюджет высокий и компания пара, то рекомендовать luxury-отдых.
7. Если длительность длинная и отдых активный, то рекомендовать экспедицию.

8. Если ни одно правило не подходит, то рекомендовать универсальный тур.

Приведенные правила демонстрируют основные типы логических переходов в системе: от климатического и поведенческого профиля к типу тура, от ресурсных ограничений к уровню сервиса, от социального состава группы к формату программы. За счет такого распределения знаний обеспечивается интерпретируемый и практически применимый результат консультации при сохранении управляемой сложности базы [2,5].

Таким образом, сформированная база знаний обеспечивает одновременно полноту рекомендаций, их предметную обоснованность и объяснимость на уровне отдельных правил. Следующим логическим этапом является формальная и прикладная проверка согласованности этой базы, включая анализ конфликтных срабатываний и устойчивости вывода при различных наборах входных фактов.

3 Проверка непротиворечивости

Проверка непротиворечивости в данном проекте понимается как анализ самой базы продукционных правил после их составления, а не как тестирование программного кода. Цель этапа состоит в том, чтобы подтвердить логическую согласованность знаний, исключить конкурирующие рекомендации при одинаковых предпосылках и обеспечить детерминированность вывода для пользовательского профиля.

В качестве объекта анализа использован полный набор из 50 продукционных правил с формой «ЕСЛИ (условие), ТО (рекомендация)». Проверка выполнялась на уровне формальной структуры правил и на уровне их прикладного смысла в контексте туристского консультирования.

3.1 Методы проверки

Для анализа непротиворечивости применялся комбинированный подход, включающий формально-логические и содержательные процедуры.

Первый метод: нормализация условий и заключений. Все правила приведены к единому представлению вида:

- множество атомарных условий по слотам (`climate`, `travel_type`, `budget`, `trip_days`, `companions`);
- одно целевое заключение в виде текстовой рекомендации.

Нормализация необходима для сравнения правил между собой. На этом шаге устраняются чисто лингвистические различия формулировок, а также фиксируются одинаковые по смыслу условия, записанные в разном порядке.

Второй метод: поиск формальных конфликтов. Под конфликтом рассматривалась ситуация, когда:

- два правила имеют одинаковую левую часть (одинаковый набор условий), но разные заключения;
- одно правило логически включает другое по условиям, однако выводит заключение, конкурирующее с более общим правилом без явного приоритета;

– группа правил дает разные рекомендации для одного и того же фактического профиля пользователя, а правило разрешения конфликта отсутствует.

Третий метод: сценарный анализ. Для проверки прикладной устойчивости формировались типовые векторы фактов (климат, тип отдыха, бюджет, длительность, состав группы), после чего оценивалось, сколько правил потенциально срабатывают одновременно и согласованы ли их рекомендации между собой. Этот шаг позволил выявить не только строгие формальные противоречия, но и практические коллизии выбора.

Четвертый метод: семантическая классификация рекомендаций. Заключения правил были распределены по функциональным классам (тип тура, уровень сервиса, география, длительность). Если рекомендации из разных правил относились к одному классу, но предлагали взаимоисключающие варианты при совпадающих условиях, фиксировалась семантическая противоречивость.

Пятый метод: оценка полноты и устойчивости. Отдельно проверялось наличие правила по умолчанию и отсутствие пустых зон, в которых пользовательский профиль может не получить заключение. Такой анализ не заменяет выявление конфликтов, но позволяет оценить надежность базы знаний в реальной консультации.

3.2 Результаты анализа

Проведенная проверка показала, что база знаний в целом покрывает широкий диапазон пользовательских сценариев, однако содержит набор конфликтов, которые влияют на однозначность вывода.

Ключевой результат: обнаружены три прямых формальных противоречия, где пары правил имеют идентичные условия и разные рекомендации.

1. Противоречие 1: правило 7 и правило 30. Условия в обоих случаях: «отдых спокойный» и «бюджет низкий». Заключения различаются: «отдых внутри страны» против «дачный отдых».

2. Противоречие 2: правило 6 и правило 29. Условия в обоих случаях: «отдых активный» и «бюджет высокий». Заключения различаются: «экстремальный тур» против «сафари».
3. Противоречие 3: правило 2 и правило 39. Условия в обоих случаях: «климат теплый» и «бюджет высокий». Заключения различаются: «курорт за границей» против «островной отдых».

Наличие таких пар означает, что при одинаковом наборе фактов система потенциально может выдавать разные рекомендации. Без дополнительного механизма разрешения конфликта итог будет зависеть не от знаний предметной области, а от технического порядка обработки правил, что методически некорректно для экспертной системы.

Кроме прямых противоречий выявлены структурные источники неоднозначности:

1. Пересечение общих и частных правил без явной иерархии. Например, одно правило задает общий тип отдыха, другое при том же профиле задает узкий формат, но порядок выбора не формализован.
2. Неоднородность шкал длительности. Одновременно используются словесные категории («короткая», «длинная») и пороговые формулировки («до 3 дней», «больше 7 дней», «больше 10 дней»), что повышает риск несовпадения интерпретаций.
3. Разный уровень детализации рекомендаций. Часть заключений описывает класс тура, часть — конкретный маршрутный формат; при совместном срабатывании это усложняет выбор единственного финального ответа.

При этом положительным результатом является то, что база знаний не имеет «пустых» состояний: предусмотрено универсальное заключение на случай, если специализированные правила не сработают. Следовательно, основной риск связан не с отсутствием ответа, а с конкуренцией ответов.

3.3 Исправление противоречий

Для устранения выявленных коллизий предлагается сочетание двух подходов: локальная переработка конфликтующих правил и внедрение формального механизма разрешения конфликтов.

3.3.1 Локальная переработка трех конфликтных пар

1. Пара 7/30 («спокойный отдых + низкий бюджет»)

Текущее состояние:

- правило 7: рекомендовать отдых внутри страны;
- правило 30: рекомендовать дачный отдых.

Предлагаемое исправление:

- сохранить правило 7 как более общее: «спокойный отдых + низкий бюджет -> бюджетный внутренний тур»;
- уточнить правило 30 дополнительными условиями, например по длительности и логистике: «спокойный отдых + низкий бюджет + длительность до 3 дней -> дачный или пригородный формат».

Результат: конкуренция заменяется отношением «общее правило / частный случай».

2. Пара 6/29 («активный отдых + высокий бюджет»)

Текущее состояние:

- правило 6: рекомендовать экстремальный тур;
- правило 29: рекомендовать сафари.

Предлагаемое исправление:

- правило 6 оставить как обобщенный класс активного премиального туризма;
- правило 29 сделать специализированным подмножеством, добавив контекстные условия (например, теплый климат и длительность больше 7 дней).

Результат: «сафари» становится частным вариантом внутри класса «экстремальный тур», а не альтернативой при тех же фактах.

3. Пара 2/39 («теплый климат + высокий бюджет»)

Текущее состояние:

- правило 2: рекомендовать курорт за границей;
- правило 39: рекомендовать островной отдых.

Предлагаемое исправление:

- правило 2 интерпретировать как общий географический класс;
- правило 39 ограничить дополнительным условием (например, длительная поездка или формат «пара/друзья»), чтобы перевести его в более специфичный сценарий.

Результат: исключается ситуация равноправных, но разных рекомендаций при одинаковой левой части.

3.3.2 Механизм разрешения конфликтов

Даже после локальной правки целесообразно закрепить единый механизм conflict resolution на уровне базы знаний. Рекомендуется следующая схема:

1. Приоритет по специфичности: правило с большим числом условий имеет более высокий ранг.
2. Приоритет по экспертной значимости: для правил с одинаковой специфичностью задается числовой приоритет.
3. Детерминированный tie-break: при равенстве ранга используется фиксированное правило выбора (например, порядок в реестре или отдельный коэффициент полезности).

Такая трехуровневая схема обеспечивает воспроизводимость вывода и предотвращает случайный выбор рекомендации.

3.3.3 Дополнительные меры стабилизации базы знаний

Для долгосрочной устойчивости базы знаний рекомендуется:

1. Ввести регламент «одно и то же условие -> одно базовое заключение». Альтернативы допускаются только после добавления дифференцирующих условий.
2. Нормировать шкалы длительности и бюджета в единой форме (например, только интервалы и пороги), чтобы исключить неоднозначные трактовки.

3. Поддерживать версионируемый журнал изменений правил с обязательной фиксацией причины изменения и ожидаемого эффекта на вывод.
4. Проводить повторную логическую ревизию после каждого расширения базы знаний, а не только в финальной стадии.

Внедрение указанных исправлений позволит перевести базу знаний из состояния частичной конкуренции рекомендаций в состояние управляемой и детерминированной продукционной системы, где логика вывода задается экспертными принципами, а не побочными эффектами порядка обработки правил.

4 Средства разработки

Для реализации прототипа экспертной системы использован практико-ориентированный стек средств разработки, обеспечивающий быструю итерацию правил, прозрачность логики вывода и воспроизводимость результата. Выбор инструментов выполнялся с учетом учебного формата проекта, требований к explain-режиму и необходимости сопровождения базы знаний командой разработчиков.

4.1 Обоснование выбора языка программирования

Базовым языком реализации выбран Python. Первое основание выбора связано с выразительностью языка при описании предметных фактов и условий правил. Для экспертной системы критично, чтобы структура знаний оставалась читаемой в исходном коде и легко ревизовалась при расширении базы правил. Синтаксис Python и поддержка декларативных конструкций позволяют формулировать правила и метаданные в форме, близкой к естественной логике предметной области.

Второе основание — технологическая совместимость со стеком веб-части и вспомогательных модулей. Python обеспечивает единый контур для формы ввода, слоя нормализации фактов, движка вывода и сохранения результатов консультации. За счет этого снижаются интеграционные издержки: не требуется разделять решение между разными языковыми платформами и согласовывать межъязыковой обмен данными.

Третье основание — темп разработки и сопровождения. Для учебного прототипа важна короткая итерация «изменение правила -> локальная проверка -> анализ результата». В Python этот цикл минимален благодаря компактной записи бизнес-логики, отсутствию избыточного шаблонного кода и широкому набору стандартных инструментов тестирования.

Четвертое основание — поддерживаемость при командной работе. В проекте база правил регулярно уточняется, а значит, код должен быть понятен разработчику, который не является автором исходного правила. В данном аспекте Python обеспечивает низкий порог входа, предсказуемую структуру модулей и простой формат ревью изменений.

Пятое основание — приемлемый баланс производительности и сложности. Хотя интерпретируемый язык уступает низкоуровневым решениям в скорости, для прототипа с ограниченным числом правил и однопользовательским сценарием это не является критическим ограничением. Приоритет отдан не максимальной пропускной способности, а прозрачности вывода и скорости модификации базы знаний.

Отдельно учитывался риск совместимости библиотек для продукционных правил с актуальной версией Python. Выбранная архитектура допускает локальный compat-слой, что позволяет сохранить преимущества выбранного языка без перехода на более сложный технологический контур.

Итоговое обоснование: Python оптимален для данного этапа жизненного цикла системы, поскольку обеспечивает архитектурную целостность, управляемую сложность и высокую скорость эволюции правил при сохранении читаемости и тестопригодности.

4.2 Описание используемых библиотек

В проекте применены минимально необходимые внешние библиотеки, каждая из которых решает четко определенную задачу в архитектуре прототипа.

Таблица 2 — Используемые средства разработки и их роль в прототипе

Компонент	**Назначение в проекте**	**Причина выбора**	**Ограничения**
Python 3.14	Базовый язык реализации приложения, правил и тестов.	Единая среда разработки для веб-слоя, движка вывода и обработки данных.	Интерпретируемая модель исполнения, умеренная вычислительная производите
Experta 1.9.4	Реализация продукционного вывода через Fact, Rule, механизм прямого и обратного рассуждения.	Понятный Python-DSL правил и простая интеграция в существующую структуру приложения.	Не самый новый стек, требуется контроль совместимости с современным Python.
Flask 3.1.3	Веб-каркас одностраничного интерфейса консультации и маршрутизации запросов.	Легковесность, прозрачная структура, удобен для прототипа без избыточной инфраструктуры.	Базовый уровень абстракции, сложные сценарии требуют ручной архитектурной дисциплины.
Flask-WTF 1.2.2	Формирование и валидация веб-формы	Снижение количества шаблонного кода и стандартизация	Зависимость от структуры формы и аккуратной

Компонент	Назначение в проекте	Причина выбора	Ограничения
	пользовательских фактов.	валидации входных данных.	синхронизации с реестром фактов.
unittest	Модульное и интеграционное тестирование маршрута, правил и сохранения сессий.	Входит в стандартную библиотеку, не требует дополнительной инфраструктуры для запуска тестов.	Менее лаконичен по синтаксису, чем специализиро фреймворки тестирования.

Сформированный набор библиотек соответствует целям проекта: минимальный технологический риск, достаточная функциональность для продукционного вывода, а также возможность расширять систему без радикальной смены стека.

4.3 Инструменты разработки

Организация процесса разработки основана на стандартных и воспроизводимых практиках:

1. Изоляция окружения через локальное виртуальное окружение `venv`, что обеспечивает стабильность зависимостей и повторяемость запусков.
2. Фиксация внешних зависимостей в `requirements.txt` для синхронизации среды между участниками команды.
3. Контроль версий в Git с поэтапной фиксацией изменений правил, формы и логики обработки.
4. Локальный запуск приложения в режиме быстрой проверки (`python -m app`) для короткого цикла обратной связи.
5. Регрессионная проверка тестами (`python -m unittest discover -s tests -v`) после изменения правил и структуры фактов.

Такой набор инструментов обеспечивает согласованный контур: изменения в базе знаний оперативно проверяются на уровне поведения прототипа и фиксируются в системе контроля версий вместе с документацией.

5 Техническое решение

5.1 Архитектура системы

Техническая реализация прототипа выполнена как монолитное веб-приложение на Flask с встроенным продукционным модулем на `experta`. Архитектура ориентирована на короткий и объяснимый цикл консультации: ввод фактов пользователем, прямой вывод, обратная проверка решения и сохранение результата в формате JSON.

Структура приложения организована по функциональным слоям:

1. Слой знаний (`app/knowledge.py`): единый реестр `TRAVEL_FACTS`, содержащий спецификации полей формы, ограничений ввода и факт-слотов для движка вывода.
2. Слой формы (`app/form_factory.py`, `app/forms.py`): schema-driven генерация `LandingForm` и преобразование пользовательского ввода в два типа payload (для вывода и для сохранения сессии).
3. Слой вывода (`app/rules.py`): набор продукционных правил, приоритеты конфликтов (`priority`/`salience`), реализация прямого и обратного вывода, explain-структуры.
4. Веб-слой (`app/__init__.py`, шаблоны): маршруты `/` и `/test`, валидация формы, запуск механизма вывода и отображение результата.
5. Слой хранения (`app/session_store.py`): файловая персистентность сеанса в `consultation/*.json`.

Такое разделение обеспечивает независимость эволюции компонентов: изменение состава фактов в реестре автоматически влияет на форму и на входной слой движка, а правила расширяются без модификации маршрутов.

5.2 Схема работы

Функциональная схема работы прототипа построена вокруг одного основного пользовательского сценария маршрута `/`:

1. Пользователь открывает страницу консультации (`GET /`), система создает объект `LandingForm` и рендерит поля ввода.

2. После отправки формы (`POST /`) выполняется `validate_on_submit()`.
3. При ошибках валидации форма повторно отображается с диагностикой по полям.
4. При корректном вводе формируется фактологический payload (`get_evaluation_input(form)`).
5. На полученных фактах выполняется прямой вывод `evaluate(..., explain=true)`.
6. Для объяснения решения выполняется обратный вывод `backward(goal: "*", ..., explain: true)`.
7. Собирается итоговый сеанс (ввод, рекомендация, explain.forward, explain.backward) и сохраняется в JSON.
8. Пользователю отображается карточка рекомендации на той же странице.
9. Кнопка «Новая консультация» возвращает в исходное состояние формы (`GET /`).

В интерфейсе предусмотрены два взаимоисключающих состояния:

1. Режим ввода (`recommendation_text == none`): показ формы.
2. Режим результата (`recommendation_text != none`): показ карточки рекомендации и кнопки повторного запуска.

Дополнительно реализован маршрут `/test`, предназначенный для демонстрации диагностического режима: система отображает совпавшие правила, шаги обратного вывода, цепочки доказательства и полный список зарегистрированных правил с приоритетами.

5.3 Алгоритм реализации

Алгоритм системы состоит из двух взаимосвязанных механизмов вывода.

Прямой вывод (forward chaining) реализован на `experta` и работает по цепочке:

1. Нормализация фактов: преобразование типов и исключение пустых значений.

2. Сброс состояния движка и рабочей памяти (`reset_runtime_state`, `reset`).
3. Декларация факта `TravelInput`.
4. Запуск `engine.run()` с формированием активаций правил через RETE-сеть.
5. Фиксация всех сработавших правил (`matched_rules`) и выбор правила с максимальным приоритетом.
6. Формирование `EvaluationResult`: рекомендация, выбранное правило, время, количество проходов и шаги explain-трассировки.

Обратный вывод (backward chaining) реализован отдельным алгоритмом в `TravelRuleEngine.backward(...)`:

1. Задается цель (`goal = "*"`) и список известных фактов.
2. Формируется упорядоченный список кандидатов по убыванию приоритета.
3. Для каждого кандидата проверяются все условия (`slot`, `operator`, `expected`) на соответствие фактам.
4. В explain-режиме накапливаются шаги доказательства (`prove-goal`, `try-rule`, `prove-condition`, `rule-proved`/`rule-failed`).
5. Первый успешно доказанный кандидат фиксируется как `selected_rule`.
6. Если кандидат не доказан, применяется fallback-правило `default-recommendation`.
7. Возвращается `BackwardResult` с полями `achieved`, `matched_rules`, `steps`, `proof`.

Внутренний механизм `experta` в контуре прямого вывода опирается на классическую схему `KnowledgeEngine -> ReteMatcher -> Agenda -> Activation -> выполнение RHS-метода правила`. Разрешение конфликтов выполняется стратегией глубинного порядка (`DepthStrategy`) по ключу `salience` и контексту фактов. В текущем проекте приоритеты `salience` синхронизированы с прикладным полем `priority`, что делает выбор правила детерминированным и прозрачным для explain-режима.

6 Тестирование прототипа

ЗАКЛЮЧЕНИЕ

В ходе работы были изучены принципы построения экспертных систем в заданной предметной области. Предметной областью прототипа экспертной системы «Консультант по путешествиям» является туризм. В результате анализа лиитератры, была составлена база знаний, сформированы продукционные правила.

Для реализации системы был выбран язык программированния Python и библиотеки. Система представляет собой веб-приложение на фреймворке Flask в которое встроена экспертная система на библиотеке experta.

В систему добавлены продукционные правила, которые позволяют получить консультацию по путешествиям. Ввод системы это форма с полями, которые являются фактами или критериями. Вывод системы карточка с рекомендацией путешествия.

Данная система органичена, только теми рекомендациями, которые указаны в продукционных правилах. Рекомендации имеют общий характер, без предложения конкретных туров, путевок.

Данную систему можно улучшить, например изменив рекомендации в правилах. Если рекомендации представить в виде объетов: путевка, тур, экскурсия и др. Можно интегрировать систему с сервисом, который предоставляет такие данные(например, по API).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Кондратьева А.А. Основы туризма: учебное пособие (практикум) [Электронный ресурс]. Ставрополь: Северо-Кавказский федеральный университет, 2021. С. 111. URL: https://www.iprbookshop.ru/135718.html (дата обращения: 30.03.2026).

2. Очилова Х.Ф., Амонбоев М. Маркетинг туризма: учебник [Электронный ресурс]. Москва: Директ-Медиа, 2022. С. 176. URL: https://biblioclub.ru/index.php?page=book&id=689008 (дата обращения: 04.04.2026).

3. Абдель Вахед Э.А.М., Мендельсон В.А., Абуталипова Л.Н. Современные технологии в индустрии туризма и гостиничном сервисе: учебник [Электронный ресурс]. Казань: Казанский национальный исследовательский технологический университет (КНИТУ), 2021. С. 204. URL: https://biblioclub.ru/index.php?page=book&id=701229 (дата обращения: 04.04.2026).

4. Шаруненко Ю.М. Рекреационный туризм: учебно-методическое пособие [Электронный ресурс]. Орел: Межрегиональная Академия безопасности и выживания (МАБИВ), 2014. С. 102. URL: https://www.iprbookshop.ru/33439.html (дата обращения: 30.03.2026).

5. Киреева Ю.А. Виды туризма: учебное пособие [Электронный ресурс]. Москва: Директ-Медиа, 2022. С. 324. URL: https://biblioclub.ru/index.php?page=book&id=686780 (дата обращения: 04.04.2026).

ПРИЛОЖЕНИЕ А

Исходный код прототипа экспертной системы

```
from __future__ import annotations

import os
from datetime import datetime, timezone

from flask import Flask, render_template, request
from flask_wtf.csrf import CSRFProtect

from app.forms import (
    LandingForm,
    SUBMIT_FIELD_NAME,
    VISIBLE_FORM_FIELDS,
    get_evaluation_input,
    get_session_input,
)
from app.prototype_testing import (
    build_prototype_testing_context,
    get_prototype_scenario_previews,
)
from app.rules import EvaluationResult, TravelRuleEngine,
init_rule_engine
from app.session_store import get_consultation_dir,
save_consultation_session

csrf = CSRFProtect()


def create_app() -> Flask:
    app = Flask(__name__)
    app.config["SECRET_KEY"] = os.getenv("SECRET_KEY", "dev-local-
secret")
    app.config["DEBUG"] = False
    app.config["WTF_CSRF_ENABLED"] = True

    csrf.init_app(app)
    init_rule_engine(app)
    get_consultation_dir()

    @app.route("/", methods=["GET", "POST"])
```

```
    def index() -> str:
        form = LandingForm()
        recommendation_text = None

        if form.validate_on_submit():
            engine = app.extensions["expert_engine"]
            if isinstance(engine, TravelRuleEngine):
                evaluation_input = get_evaluation_input(form)
                evaluation_result = engine.evaluate(evaluation_input,
explain=True)

                if isinstance(evaluation_result, EvaluationResult):
                    recommendation_text =
evaluation_result.recommendation
                    backward_result = engine.backward(
                        goal="*",
                        known_facts=evaluation_input,
                        explain=True,
                    )
                    save_consultation_session(
                        {
                            "created_at_utc":
datetime.now(timezone.utc).isoformat(),
                            "input": get_session_input(form),
                            "recommendation": recommendation_text,
                            "explain": {
                                "forward": {
                                    "matched_rules":
list(evaluation_result.matched_rules),
                                    "selected_rule":
evaluation_result.selected_rule,
                                    "elapsed_ms":
evaluation_result.elapsed_ms,
                                    "passes":
evaluation_result.passes,
                                    "steps":
list(evaluation_result.steps),
                                },
                                "backward": {
                                    "goal": backward_result.goal,
                                    "achieved":
backward_result.achieved,
                                    "selected_rule":
```

```
backward_result.selected_rule,
                                        "matched_rules":
list(backward_result.matched_rules),
                                        "recommendation":
backward_result.recommendation,
                                        "elapsed_ms":
backward_result.elapsed_ms,
                                        "passes": backward_result.passes,
                                        "steps":
list(backward_result.steps),
                                        "proof": backward_result.proof,
                                    },
                                },
                            }
                        )

        return render_template(
            "index.html",
            form=form,
            visible_fields=VISIBLE_FORM_FIELDS,
            submit_field_name=SUBMIT_FIELD_NAME,
            recommendation_text=recommendation_text,
        )

    @app.route("/test", methods=["GET"])
    def test_route() -> str:
        run_tests = request.args.get("run_tests") == "1"
        engine = app.extensions["expert_engine"]
        if not isinstance(engine, TravelRuleEngine):
            return render_template(
                "test.html",
                engine_error=True,
                run_tests=run_tests,
                scenario_previews=get_prototype_scenario_previews(),
            )

        test_context: dict[str, object] = {}
        if run_tests:
            test_context = build_prototype_testing_context(engine)

        return render_template(
            "test.html",
            engine_error=False,
```

```
            run_tests=run_tests,
            rules_count=engine.rules_count(),
            scenario_previews=get_prototype_scenario_previews(),
            **test_context,
        )

    return app
```

Листинг А.1 — Код веб приложения

```
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Literal

ValidatorKind = Literal["required", "number_range"]
FieldType = Literal["string", "integer", "select", "textarea",
"submit"]
Operator = Literal["eq", "lt", "lte", "gt", "gte"]


@dataclass(frozen=True)
class ValidatorSpec:
    kind: ValidatorKind
    message: str | None = None
    min_value: int | None = None
    max_value: int | None = None


@dataclass(frozen=True)
class FactSpec:
    name: str
    label: str
    field_type: FieldType
    required: bool = False
    choices: tuple[tuple[str, str], ...] = ()
    validators: tuple[ValidatorSpec, ...] = ()
    ui: dict[str, Any] = field(default_factory=dict)
    fact_slot: str | None = None
    include_in_session: bool = True


@dataclass(frozen=True)
```

```
class ConditionSpec:
    slot: str
    op: Operator
    value: str | int


TRAVEL_FACTS: tuple[FactSpec, ...] = (
    FactSpec(
        name="season",
        label="Сезон поездки",
        field_type="select",
        required=True,
        choices=(
            ("any", "Без предпочтений"),
            ("spring", "Весна"),
            ("summer", "Лето"),
            ("autumn", "Осень"),
            ("winter", "Зима"),
        ),
        validators=(
            ValidatorSpec(kind="required", message="Выберите сезон
поездки."),
        ),
        fact_slot="season",
    ),
    FactSpec(
        name="hobby",
        label="Увлечение",
        field_type="select",
        required=True,
        choices=(
            ("hiking", "Походы"),
            ("museum", "Музеи"),
            ("food", "Гастрономия"),
            ("any", "Без предпочтений"),
        ),
        validators=(
            ValidatorSpec(kind="required", message="Выберите
увлечение."),
        ),
        fact_slot="hobby",
    ),
    FactSpec(
```

```
        name="budget_rub",
        label="Бюджет (руб.)",
        field_type="integer",
        required=True,
        validators=(
            ValidatorSpec(kind="required", message="Укажите
бюджет."),
            ValidatorSpec(
                kind="number_range",
                min_value=1000,
                message="Бюджет должен быть не меньше 1000 руб.",
            ),
        ),
        ui={"min": 1000, "step": 1000, "placeholder": "Например:
120000"},
        fact_slot="budget_rub",
    ),
    FactSpec(
        name="trip_days",
        label="Длительность (дней)",
        field_type="integer",
        required=True,
        validators=(
            ValidatorSpec(kind="required", message="Укажите
длительность поездки."),
            ValidatorSpec(
                kind="number_range",
                min_value=1,
                max_value=60,
                message="Допустимый диапазон: от 1 до 60 дней.",
            ),
        ),
        ui={"min": 1, "max": 60, "placeholder": "Например: 7"},
        fact_slot="trip_days",
    ),
    FactSpec(
        name="climate",
        label="Предпочитаемый климат",
        field_type="select",
        required=True,
        choices=(
            ("any", "Без предпочтений"),
            ("warm", "Теплый"),
```

```
            ("mild", "Умеренный"),
            ("cold", "Прохладный"),
        ),
        validators=(
            ValidatorSpec(kind="required", message="Выберите
климат."),
        ),
        fact_slot="climate",
    ),
    FactSpec(
        name="travel_type",
        label="Тип отдыха",
        field_type="select",
        required=True,
        choices=(
            ("relax", "Спокойный"),
            ("active", "Активный"),
            ("mixed", "Смешанный"),
            ("culture", "Культурный"),
            ("health", "Оздоровительный"),
            ("business", "Деловой"),
            ("eco", "Экологический"),
            ("education", "Обучающий"),
        ),
        validators=(
            ValidatorSpec(kind="required", message="Выберите тип
отдыха."),
        ),
        fact_slot="travel_type",
    ),
    FactSpec(
        name="companions",
        label="Состав поездки",
        field_type="select",
        required=True,
        choices=(
            ("solo", "Один"),
            ("couple", "Пара"),
            ("family", "Семья"),
            ("friends", "Друзья"),
        ),
        validators=(
            ValidatorSpec(kind="required", message="Выберите состав
```

```
поездки."),
        ),
        fact_slot="companions",
    ),
    FactSpec(
        name="service_level",
        label="Уровень сервиса",
        field_type="select",
        required=True,
        choices=(
            ("economy", "Эконом"),
            ("standard", "Стандарт"),
            ("premium", "Премиум"),
        ),
        validators=(
            ValidatorSpec(kind="required", message="Выберите уровень
сервиса."),
        ),
        fact_slot="service_level",
    ),
    FactSpec(
        name="visa_mode",
        label="Визовые ограничения",
        field_type="select",
        required=True,
        choices=(
            ("any", "Не важно"),
            ("visa_free_only", "Только без визы"),
            ("visa_ready", "Готов(а) оформить визу"),
        ),
        validators=(
            ValidatorSpec(kind="required", message="Укажите визовые
ограничения."),
        ),
        fact_slot="visa_mode",
    ),
    FactSpec(
        name="insurance",
        label="Страховка",
        field_type="select",
        required=True,
        choices=(
            ("yes", "Страховка оформлена или будет оформлена"),
```

```
            ("no", "Без страховки"),
        ),
        validators=(
            ValidatorSpec(
                kind="required",
                message="Укажите, готовы ли вы оформить страховку.",
            ),
        ),
        fact_slot="insurance",
    ),
    FactSpec(
        name="notes",
        label="Дополнительные пожелания",
        field_type="textarea",
        ui={"placeholder": "Например: нужен пляж, мало пересадок,
прямой рейс."},
    ),
    FactSpec(
        name="submit",
        label="Начать консультацию",
        field_type="submit",
        include_in_session=False,
    ),
)

DEFAULT_RECOMMENDATION = (
    "Рекомендуется универсальный экскурсионный отдых по вашему
бюджету и срокам."
)
```

Листинг А.2 — Добавление фактов

```
from __future__ import annotations

from dataclasses import dataclass
from time import perf_counter
from typing import TYPE_CHECKING, Any, Callable, Literal, Mapping

from app.experta_compat import patch_experta_compat
from app.knowledge import ConditionSpec, DEFAULT_RECOMMENDATION,
TRAVEL_FACTS

patch_experta_compat()
```

```
from experta import MATCH, TEST, Fact, KnowledgeEngine, Rule  # noqa:
E402

if TYPE_CHECKING:
    from flask import Flask


DEFAULT_RULE_NAME = "default-recommendation"


@dataclass(frozen=True)
class RuleMetadata:
    name: str
    priority: int
    recommendation: str
    conditions: tuple[ConditionSpec, ...]


@dataclass(frozen=True)
class EvaluationResult:
    recommendation: str
    matched_rules: tuple[str, ...]
    selected_rule: str
    elapsed_ms: float
    passes: int
    steps: tuple[dict[str, Any], ...] = ()


@dataclass(frozen=True)
class BackwardResult:
    goal: str
    achieved: bool
    selected_rule: str | None
    matched_rules: tuple[str, ...]
    recommendation: str | None
    elapsed_ms: float
    passes: int
    steps: tuple[dict[str, Any], ...]
    proof: dict[str, Any] | None = None


def _register_rule(
```

```
    *,
    name: str,
    priority: int,
    recommendation: str,
    conditions: tuple[ConditionSpec, ...],
) -> Callable[[Callable[..., Any]], Callable[..., Any]]:
    metadata = RuleMetadata(
        name=name,
        priority=priority,
        recommendation=recommendation,
        conditions=conditions,
    )

    def decorator(rule_callable: Callable[..., Any]) -> Callable[...,
Any]:
        # Keep metadata priority and experta conflict priority in
sync.
        setattr(rule_callable, "salience", priority)
        setattr(rule_callable, "_travel_rule_metadata", metadata)
        return rule_callable

    return decorator


def _collect_rule_metadata(engine_class: type[KnowledgeEngine]) ->
dict[str, RuleMetadata]:
    metadata_by_name: dict[str, RuleMetadata] = {}
    for attr_name in dir(engine_class):
        attr = getattr(engine_class, attr_name)
        metadata = getattr(attr, "_travel_rule_metadata", None)
        if isinstance(metadata, RuleMetadata):
            metadata_by_name[metadata.name] = metadata

    metadata_by_name[DEFAULT_RULE_NAME] = RuleMetadata(
        name=DEFAULT_RULE_NAME,
        priority=-1000,
        recommendation=DEFAULT_RECOMMENDATION,
        conditions=(),
    )
    return metadata_by_name


def _sorted_rules(metadata: Mapping[str, RuleMetadata]) ->
```

```
list[RuleMetadata]:
    return sorted(
        (
            item
            for item in metadata.values()
            if item.name != DEFAULT_RULE_NAME
        ),
        key=lambda item: item.priority,
        reverse=True,
    )


def _apply_operator(op: Literal["eq", "lt", "lte", "gt", "gte"],
left: Any, right: Any) -> bool:
    if op == "eq":
        return left == right
    if op == "lt":
        return left < right
    if op == "lte":
        return left <= right
    if op == "gt":
        return left > right
    if op == "gte":
        return left >= right
    raise ValueError(f"Unsupported operator: {op}")


def _condition_is_satisfied(condition: ConditionSpec, known_facts:
Mapping[str, Any]) -> bool:
    if condition.slot not in known_facts:
        return False
    return _apply_operator(condition.op, known_facts[condition.slot],
condition.value)


class TravelInput(Fact):
    """Input facts for travel recommendation inference."""


class _TravelExpertEngine(KnowledgeEngine):
    def __init__(self) -> None:
        super().__init__()
        self.rule_metadata = _collect_rule_metadata(self.__class__)
```

```
        self.reset_runtime_state()

    def reset_runtime_state(self) -> None:
        self.matched_rules: list[str] = []
        self.selected_rule: str | None = None
        self.selected_priority = float("-inf")
        self.recommendation = DEFAULT_RECOMMENDATION

    def register_match(self, rule_name: str) -> None:
        metadata = self.rule_metadata[rule_name]
        self.matched_rules.append(rule_name)
        if metadata.priority > self.selected_priority:
            self.selected_priority = metadata.priority
            self.selected_rule = metadata.name
            self.recommendation = metadata.recommendation

    @_register_rule(
        name="warm-relax-premium",
        priority=340,
        recommendation=(
            "Пляжный отдых в теплой стране с повышенным уровнем
комфорта: "
            "например, Мальдивы, Сейшелы, ОАЭ"
        ),
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="warm"),
            ConditionSpec(slot="travel_type", op="eq",
value="relax"),
            ConditionSpec(slot="budget_rub", op="gte", value=100000),
        ),
    )
    @Rule(
        TravelInput(climate="warm", travel_type="relax",
budget_rub=MATCH.budget),
        TEST(lambda budget: budget >= 100000),
    )
    def rule_warm_relax_premium(self, budget: int) -> None:  # noqa:
ARG002
        self.register_match("warm-relax-premium")

    @_register_rule(
        name="summer-family-beach",
        priority=338,
```

```
        recommendation=(
            "Семейный летний пляжный отдых с лёгким графиком и
короткими переездами: "
            "например, Турция, Болгария, Крым"
        ),
        conditions=(
            ConditionSpec(slot="season", op="eq", value="summer"),
            ConditionSpec(slot="climate", op="eq", value="warm"),
            ConditionSpec(slot="companions", op="eq",
value="family"),
        ),
    )
    @Rule(TravelInput(season="summer", climate="warm",
companions="family"))
    def rule_summer_family_beach(self) -> None:
        self.register_match("summer-family-beach")

    @_register_rule(
        name="winter-active-ski",
        priority=337,
        recommendation=(
            "Зимний активный тур с горнолыжным курортом или снежными
маршрутами: "
            "например, Альпы, Хибины, Шерегеш"
        ),
        conditions=(
            ConditionSpec(slot="season", op="eq", value="winter"),
            ConditionSpec(slot="climate", op="eq", value="cold"),
            ConditionSpec(slot="travel_type", op="eq",
value="active"),
        ),
    )
    @Rule(TravelInput(season="winter", climate="cold",
travel_type="active"))
    def rule_winter_active_ski(self) -> None:
        self.register_match("winter-active-ski")

    @_register_rule(
        name="family-health-insured",
        priority=336,
        recommendation=(
            "Семейная оздоровительная программа в санаторном формате:
"
```

```
            "например, Кавминводы, Белокуриха, санатории Крыма"
        ),
        conditions=(
            ConditionSpec(slot="companions", op="eq",
value="family"),
            ConditionSpec(slot="travel_type", op="eq",
value="health"),
            ConditionSpec(slot="insurance", op="eq", value="yes"),
        ),
    )
    @Rule(TravelInput(companions="family", travel_type="health",
insurance="yes"))
    def rule_family_health_insured(self) -> None:
        self.register_match("family-health-insured")

    @_register_rule(
        name="business-premium-city",
        priority=335,
        recommendation=(
            "Деловая поездка в крупный город с центральным отелем и
премиальным сервисом: "
            "например, Москва, Шанхай, Сингапур"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="business"),
            ConditionSpec(slot="service_level", op="eq",
value="premium"),
            ConditionSpec(slot="budget_rub", op="gte", value=90000),
        ),
    )
    @Rule(
        TravelInput(
            travel_type="business",
            service_level="premium",
            budget_rub=MATCH.budget,
        ),
        TEST(lambda budget: budget >= 90000),
    )
    def rule_business_premium_city(self, budget: int) -> None:  #
noqa: ARG002
        self.register_match("business-premium-city")
```

```
    @_register_rule(
        name="visa-free-family-sea",
        priority=334,
        recommendation=(
            "Теплое семейное направление без визы с простой
логистикой и морским отдыхом: "
            "например, Сочи, Китай"
        ),
        conditions=(
            ConditionSpec(slot="visa_mode", op="eq",
value="visa_free_only"),
            ConditionSpec(slot="companions", op="eq",
value="family"),
            ConditionSpec(slot="climate", op="eq", value="warm"),
        ),
    )
    @Rule(
        TravelInput(
            visa_mode="visa_free_only",
            companions="family",
            climate="warm",
        ),
    )
    def rule_visa_free_family_sea(self) -> None:
        self.register_match("visa-free-family-sea")

    @_register_rule(
        name="eco-spring-hike",
        priority=333,
        recommendation=(
            "Весенний экотур с пешими маршрутами, национальными
парками и умеренной нагрузкой: "
            "например, Сочи, Алтай, Карелия"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq", value="eco"),
            ConditionSpec(slot="season", op="eq", value="spring"),
            ConditionSpec(slot="hobby", op="eq", value="hiking"),
        ),
    )
    @Rule(TravelInput(travel_type="eco", season="spring",
hobby="hiking"))
    def rule_eco_spring_hike(self) -> None:
```

```
        self.register_match("eco-spring-hike")

    @_register_rule(
        name="education-food-workshop",
        priority=332,
        recommendation=(
            "Обучающий гастрономический тур с мастер-классами и
дегустациями: "
            "например, Италия, Франция, Грузия"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="education"),
            ConditionSpec(slot="hobby", op="eq", value="food"),
        ),
    )
    @Rule(TravelInput(travel_type="education", hobby="food"))
    def rule_education_food_workshop(self) -> None:
        self.register_match("education-food-workshop")

    @_register_rule(
        name="dance-culture-festival",
        priority=331,
        recommendation=(
            "Культурная поездка на фестивали и танцевальные
мероприятия: "
            "например, Аргентина (танго), Испания (фламенко),
Бразилия (самба)"
        ),
        conditions=(
            ConditionSpec(slot="hobby", op="eq", value="dance"),
            ConditionSpec(slot="travel_type", op="eq",
value="culture"),
        ),
    )
    @Rule(TravelInput(hobby="dance", travel_type="culture"))
    def rule_dance_culture_festival(self) -> None:
        self.register_match("dance-culture-festival")

    @_register_rule(
        name="museum-culture-grand-tour",
        priority=330,
        recommendation=(
```

```
            "Насыщенный музейно-экскурсионный маршрут по нескольким
городам: "
            "например, Париж, Версаль, Санкт-Петербург, Рим"
        ),
        conditions=(
            ConditionSpec(slot="hobby", op="eq", value="museum"),
            ConditionSpec(slot="travel_type", op="eq",
value="culture"),
            ConditionSpec(slot="budget_rub", op="gte", value=85000),
        ),
    )
    @Rule(
        TravelInput(hobby="museum", travel_type="culture",
budget_rub=MATCH.budget),
        TEST(lambda budget: budget >= 85000),
    )
    def rule_museum_culture_grand_tour(self, budget: int) -> None:  #
noqa: ARG002
        self.register_match("museum-culture-grand-tour")

    @_register_rule(
        name="hiking-winter-adventure",
        priority=329,
        recommendation=(
            "Зимний приключенческий маршрут с треккингом и
сопровождением инструкторов: "
            "например, Хибины, Камчатка, Урал"
        ),
        conditions=(
            ConditionSpec(slot="hobby", op="eq", value="hiking"),
            ConditionSpec(slot="season", op="eq", value="winter"),
            ConditionSpec(slot="travel_type", op="eq",
value="active"),
        ),
    )
    @Rule(TravelInput(hobby="hiking", season="winter",
travel_type="active"))
    def rule_hiking_winter_adventure(self) -> None:
        self.register_match("hiking-winter-adventure")

    @_register_rule(
        name="couple-autumn-culture",
        priority=328,
```

```
        recommendation=(
            "Романтический культурный тур по городам для пары в
бархатный сезон: "
            "например, Прага, Будапешт, Вена"
        ),
        conditions=(
            ConditionSpec(slot="companions", op="eq",
value="couple"),
            ConditionSpec(slot="season", op="eq", value="autumn"),
            ConditionSpec(slot="travel_type", op="eq",
value="culture"),
        ),
    )
    @Rule(
        TravelInput(companions="couple", season="autumn",
travel_type="culture"),
    )
    def rule_couple_autumn_culture(self) -> None:
        self.register_match("couple-autumn-culture")

    @_register_rule(
        name="friends-winter-sport",
        priority=327,
        recommendation=(
            "Активный зимний тур для компании друзей со спортом и
насыщенным досугом: "
            "например, Шерегеш, Домбай, Красноярские столбы"
        ),
        conditions=(
            ConditionSpec(slot="companions", op="eq",
value="friends"),
            ConditionSpec(slot="season", op="eq", value="winter"),
            ConditionSpec(slot="travel_type", op="eq",
value="active"),
        ),
    )
    @Rule(
        TravelInput(companions="friends", season="winter",
travel_type="active"),
    )
    def rule_friends_winter_sport(self) -> None:
        self.register_match("friends-winter-sport")
```

```
    @_register_rule(
        name="solo-active-adventure",
        priority=326,
        recommendation=(
            "Путешествие с активной программой и присоединением к
организованным маршрутам: "
            "например, Килиманджаро, Алтай, Карелия"
        ),
        conditions=(
            ConditionSpec(slot="companions", op="eq", value="solo"),
            ConditionSpec(slot="travel_type", op="eq",
value="active"),
            ConditionSpec(slot="trip_days", op="gte", value=6),
        ),
    )
    @Rule(
        TravelInput(companions="solo", travel_type="active",
trip_days=MATCH.days),
        TEST(lambda days: days >= 6),
    )
    def rule_solo_active_adventure(self, days: int) -> None:  # noqa:
ARG002
        self.register_match("solo-active-adventure")

    @_register_rule(
        name="no-insurance-active-safe",
        priority=325,
        recommendation=(
            "Безопасный активный маршрут внутри страны без
экстремальных нагрузок: "
            "например, Подмосковье, Ленобласть, Карелия"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="active"),
            ConditionSpec(slot="insurance", op="eq", value="no"),
        ),
    )
    @Rule(TravelInput(travel_type="active", insurance="no"))
    def rule_no_insurance_active_safe(self) -> None:
        self.register_match("no-insurance-active-safe")

    @_register_rule(
```

```
        name="no-insurance-cold-safe",
        priority=324,
        recommendation=(
            "Короткая спокойная поездка без удаленных локаций: "
            "например, Кижи, Великий Устюг, Суздаль"
        ),
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="cold"),
            ConditionSpec(slot="insurance", op="eq", value="no"),
        ),
    )
    @Rule(TravelInput(climate="cold", insurance="no"))
    def rule_no_insurance_cold_safe(self) -> None:
        self.register_match("no-insurance-cold-safe")

    @_register_rule(
        name="premium-visa-ready-relax",
        priority=323,
        recommendation=(
            "Комфортный зарубежный отпуск с визой и высоким сервисом:
"
            "например, Шри-Ланка, Маврикий, Италия, Франция"
        ),
        conditions=(
            ConditionSpec(slot="visa_mode", op="eq",
value="visa_ready"),
            ConditionSpec(slot="travel_type", op="eq",
value="relax"),
            ConditionSpec(slot="service_level", op="eq",
value="premium"),
        ),
    )
    @Rule(
        TravelInput(
            visa_mode="visa_ready",
            travel_type="relax",
            service_level="premium",
        ),
    )
    def rule_premium_visa_ready_relax(self) -> None:
        self.register_match("premium-visa-ready-relax")

    @_register_rule(
```

```
        name="active-short-budget",
        priority=310,
        recommendation=(
            "Активный короткий тур по России или соседним
направлениям: "
            "например, Алтай, Адыгея, Карелия, Урал"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="active"),
            ConditionSpec(slot="budget_rub", op="lt", value=100000),
            ConditionSpec(slot="trip_days", op="lte", value=7),
        ),
    )
    @Rule(
        TravelInput(travel_type="active", budget_rub=MATCH.budget,
trip_days=MATCH.days),
        TEST(lambda budget, days: budget < 100000 and days <= 7),
    )
    def rule_active_short_budget(self, budget: int, days: int) ->
None:  # noqa: ARG002
        self.register_match("active-short-budget")

    @_register_rule(
        name="family-mild-climate",
        priority=305,
        recommendation=(
            "Семейный отдых в умеренном климате с короткими
переездами: "
            "например, Суздаль, Казань, Псков"
        ),
        conditions=(
            ConditionSpec(slot="companions", op="eq",
value="family"),
            ConditionSpec(slot="climate", op="eq", value="mild"),
        ),
    )
    @Rule(TravelInput(companions="family", climate="mild"))
    def rule_family_mild_climate(self) -> None:
        self.register_match("family-mild-climate")

    @_register_rule(
        name="budget-weekend-domestic",
```

```
        priority=295,
        recommendation=(
            "Недорогая поездка на выходные по России с простой
логистикой: "
            "например, Псков, Калуга, Ярославль"
        ),
        conditions=(
            ConditionSpec(slot="budget_rub", op="lt", value=50000),
            ConditionSpec(slot="trip_days", op="lte", value=3),
        ),
    )
    @Rule(
        TravelInput(budget_rub=MATCH.budget, trip_days=MATCH.days),
        TEST(lambda budget, days: budget < 50000 and days <= 3),
    )
    def rule_budget_weekend_domestic(self, budget: int, days: int) ->
None:  # noqa: ARG002
        self.register_match("budget-weekend-domestic")

    @_register_rule(
        name="culture-short-citybreak",
        priority=294,
        recommendation=(
            "Короткий культурный тур по городам с музеями,
экскурсиями и вечерними прогулками: "
            "например, Казань, Калининград, Нижний Новгород"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="culture"),
            ConditionSpec(slot="trip_days", op="lte", value=5),
        ),
    )
    @Rule(
        TravelInput(travel_type="culture", trip_days=MATCH.days),
        TEST(lambda days: days <= 5),
    )
    def rule_culture_short_citybreak(self, days: int) -> None:  #
noqa: ARG002
        self.register_match("culture-short-citybreak")

    @_register_rule(
        name="relax-medium-resort",
```

```
        priority=293,
        recommendation=(
            "Стандартный курортный отдых средней длительности: "
            "например, Анапа, Геленджик, Южный берег Крыма"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="relax"),
            ConditionSpec(slot="budget_rub", op="gte", value=70000),
            ConditionSpec(slot="budget_rub", op="lt", value=130000),
            ConditionSpec(slot="trip_days", op="gte", value=5),
            ConditionSpec(slot="trip_days", op="lte", value=10),
        ),
    )
    @Rule(
        TravelInput(travel_type="relax", budget_rub=MATCH.budget,
trip_days=MATCH.days),
        TEST(lambda budget, days: 70000 <= budget < 130000 and 5 <=
days <= 10),
    )
    def rule_relax_medium_resort(self, budget: int, days: int) ->
None:  # noqa: ARG002
        self.register_match("relax-medium-resort")

    @_register_rule(
        name="health-short-retreat",
        priority=292,
        recommendation=(
            "Короткая оздоровительная поездка с восстановительным
режимом: "
            "например, Кисловодск, Ессентуки, санатории Подмосковья"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="health"),
            ConditionSpec(slot="trip_days", op="lte", value=8),
        ),
    )
    @Rule(
        TravelInput(travel_type="health", trip_days=MATCH.days),
        TEST(lambda days: days <= 8),
    )
    def rule_health_short_retreat(self, days: int) -> None:  # noqa:
```

```
ARG002
        self.register_match("health-short-retreat")

    @_register_rule(
        name="business-short-standard",
        priority=291,
        recommendation=(
            "Короткая деловая поездка с четким графиком и удобным
размещением: "
            "например, Минск, Екатеринбург, Ростов-на-Дону"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="business"),
            ConditionSpec(slot="trip_days", op="lte", value=4),
        ),
    )
    @Rule(
        TravelInput(travel_type="business", trip_days=MATCH.days),
        TEST(lambda days: days <= 4),
    )
    def rule_business_short_standard(self, days: int) -> None:  #
noqa: ARG002
        self.register_match("business-short-standard")

    @_register_rule(
        name="eco-budget-trail",
        priority=290,
        recommendation=(
            "Бюджетный экотур с природными маршрутами и базовой
инфраструктурой: "
            "например, Воронежский заповедник, Калужские засеки,
Угра"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq", value="eco"),
            ConditionSpec(slot="budget_rub", op="lt", value=90000),
        ),
    )
    @Rule(
        TravelInput(travel_type="eco", budget_rub=MATCH.budget),
        TEST(lambda budget: budget < 90000),
    )
```

```
    def rule_eco_budget_trail(self, budget: int) -> None:  # noqa:
ARG002
        self.register_match("eco-budget-trail")

    @_register_rule(
        name="mixed-week-combo",
        priority=289,
        recommendation=(
            "Комбинированный тур на неделю: часть программы
спокойная, часть активная или экскурсионная: "
            "например, Сочи, Крым, Алтай"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="mixed"),
            ConditionSpec(slot="trip_days", op="gte", value=7),
            ConditionSpec(slot="trip_days", op="lte", value=10),
        ),
    )
    @Rule(
        TravelInput(travel_type="mixed", trip_days=MATCH.days),
        TEST(lambda days: 7 <= days <= 10),
    )
    def rule_mixed_week_combo(self, days: int) -> None:  # noqa:
ARG002
        self.register_match("mixed-week-combo")

    @_register_rule(
        name="long-culture-grand-tour",
        priority=288,
        recommendation=(
            "Длинный экскурсионный маршрут по нескольким городам с
насыщенной программой: "
            "например, Золотое кольцо, Узбекистан, Европа на поезде"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="culture"),
            ConditionSpec(slot="trip_days", op="gte", value=10),
            ConditionSpec(slot="budget_rub", op="gte", value=90000),
        ),
    )
    @Rule(
```

```
        TravelInput(travel_type="culture", trip_days=MATCH.days,
budget_rub=MATCH.budget),
        TEST(lambda days, budget: days >= 10 and budget >= 90000),
    )
    def rule_long_culture_grand_tour(self, days: int, budget: int) ->
None:  # noqa: ARG002
        self.register_match("long-culture-grand-tour")

    @_register_rule(
        name="long-active-expedition",
        priority=287,
        recommendation=(
            "Активное путешествие или экспедиционный маршрут: "
            "например, Камчатка, Байкал, Горный Алтай"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="active"),
            ConditionSpec(slot="trip_days", op="gte", value=12),
            ConditionSpec(slot="budget_rub", op="gte", value=110000),
        ),
    )
    @Rule(
        TravelInput(travel_type="active", trip_days=MATCH.days,
budget_rub=MATCH.budget),
        TEST(lambda days, budget: days >= 12 and budget >= 110000),
    )
    def rule_long_active_expedition(self, days: int, budget: int) ->
None:  # noqa: ARG002
        self.register_match("long-active-expedition")

    @_register_rule(
        name="couple-relax-premium",
        priority=286,
        recommendation=(
            "Премиальный романтический отдых для пары с акцентом на
комфорт и приватность: "
            "например, Баку, Мальдивы, Шри-Ланка"
        ),
        conditions=(
            ConditionSpec(slot="companions", op="eq",
value="couple"),
            ConditionSpec(slot="travel_type", op="eq",
```

```
value="relax"),
            ConditionSpec(slot="service_level", op="eq",
value="premium"),
        ),
    )
    @Rule(
        TravelInput(companions="couple", travel_type="relax",
service_level="premium"),
    )
    def rule_couple_relax_premium(self) -> None:
        self.register_match("couple-relax-premium")

    @_register_rule(
        name="friends-budget-roadtrip",
        priority=285,
        recommendation=(
            "Бюджетное дорожное путешествие или насыщенный маршрут
для компании друзей: "
            "например, Золотое кольцо, Крым на машине, Кавказ"
        ),
        conditions=(
            ConditionSpec(slot="companions", op="eq",
value="friends"),
            ConditionSpec(slot="budget_rub", op="lt", value=90000),
        ),
    )
    @Rule(
        TravelInput(companions="friends", budget_rub=MATCH.budget),
        TEST(lambda budget: budget < 90000),
    )
    def rule_friends_budget_roadtrip(self, budget: int) -> None:  #
noqa: ARG002
        self.register_match("friends-budget-roadtrip")

    @_register_rule(
        name="family-culture-schoolbreak",
        priority=284,
        recommendation=(
            "Короткая семейная культурная поездка на неделю: "
            "например, Москва, Санкт-Петербург, Казань"
        ),
        conditions=(
            ConditionSpec(slot="companions", op="eq",
```

```
value="family"),
            ConditionSpec(slot="travel_type", op="eq",
value="culture"),
            ConditionSpec(slot="trip_days", op="lte", value=7),
        ),
    )
    @Rule(
        TravelInput(companions="family", travel_type="culture",
trip_days=MATCH.days),
        TEST(lambda days: days <= 7),
    )
    def rule_family_culture_schoolbreak(self, days: int) -> None:  #
noqa: ARG002
        self.register_match("family-culture-schoolbreak")

    @_register_rule(
        name="warm-summer-beach",
        priority=283,
        recommendation=(
            "Летний морской отдых в теплом климате: "
            "например, Средиземноморье, Турция, Анапа"
        ),
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="warm"),
            ConditionSpec(slot="season", op="eq", value="summer"),
        ),
    )
    @Rule(TravelInput(climate="warm", season="summer"))
    def rule_warm_summer_beach(self) -> None:
        self.register_match("warm-summer-beach")

    @_register_rule(
        name="cold-winter-relax",
        priority=282,
        recommendation=(
            "Спокойный зимний отдых в холодном климате: спа, термы
или северные отели: "
            "например, Хибины, Карелия, Лапландия"
        ),
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="cold"),
            ConditionSpec(slot="season", op="eq", value="winter"),
            ConditionSpec(slot="travel_type", op="eq",
```

```
value="relax"),
        ),
    )
    @Rule(TravelInput(climate="cold", season="winter",
travel_type="relax"))
    def rule_cold_winter_relax(self) -> None:
        self.register_match("cold-winter-relax")

    @_register_rule(
        name="mild-spring-city",
        priority=281,
        recommendation=(
            "Весенний маршрут в умеренном климате: прогулки,
экскурсии и легкая городская программа: "
            "например, Будапешт, Прага, Калининград"
        ),
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="mild"),
            ConditionSpec(slot="season", op="eq", value="spring"),
        ),
    )
    @Rule(TravelInput(climate="mild", season="spring"))
    def rule_mild_spring_city(self) -> None:
        self.register_match("mild-spring-city")

    @_register_rule(
        name="warm-autumn-gastro",
        priority=280,
        recommendation=(
            "Теплый осенний гастрономический тур с рынками и
локальными дегустациями: "
            "например, Италия, Грузия, Турция"
        ),
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="warm"),
            ConditionSpec(slot="season", op="eq", value="autumn"),
            ConditionSpec(slot="hobby", op="eq", value="food"),
        ),
    )
    @Rule(TravelInput(climate="warm", season="autumn", hobby="food"))
    def rule_warm_autumn_gastro(self) -> None:
        self.register_match("warm-autumn-gastro")
```

```
    @_register_rule(
        name="cold-short-culture",
        priority=279,
        recommendation=(
            "Короткая культурная поездка в холодный сезон без сложной
логистики: "
            "например, Архангельск, Псков, Великий Новгород"
        ),
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="cold"),
            ConditionSpec(slot="travel_type", op="eq",
value="culture"),
            ConditionSpec(slot="trip_days", op="lte", value=4),
        ),
    )
    @Rule(
        TravelInput(climate="cold", travel_type="culture",
trip_days=MATCH.days),
        TEST(lambda days: days <= 4),
    )
    def rule_cold_short_culture(self, days: int) -> None:  # noqa:
ARG002
        self.register_match("cold-short-culture")

    @_register_rule(
        name="visa-free-short-trip",
        priority=278,
        recommendation=(
            "Короткое безвизовое направление, чтобы не тратить время
на документы: "
            "например, Китай, Армения, Беларусь"
        ),
        conditions=(
            ConditionSpec(slot="visa_mode", op="eq",
value="visa_free_only"),
            ConditionSpec(slot="trip_days", op="lte", value=7),
        ),
    )
    @Rule(
        TravelInput(visa_mode="visa_free_only",
trip_days=MATCH.days),
        TEST(lambda days: days <= 7),
    )
```

```
    def rule_visa_free_short_trip(self, days: int) -> None:  # noqa:
ARG002
        self.register_match("visa-free-short-trip")

    @_register_rule(
        name="visa-free-business-quick",
        priority=277,
        recommendation=(
            "Быстрая безвизовая деловая поездка с минимальными
рисками: "
            "например, Ереван, Белград, Тбилиси"
        ),
        conditions=(
            ConditionSpec(slot="visa_mode", op="eq",
value="visa_free_only"),
            ConditionSpec(slot="travel_type", op="eq",
value="business"),
            ConditionSpec(slot="trip_days", op="lte", value=5),
        ),
    )
    @Rule(
        TravelInput(
            visa_mode="visa_free_only",
            travel_type="business",
            trip_days=MATCH.days,
        ),
        TEST(lambda days: days <= 5),
    )
    def rule_visa_free_business_quick(self, days: int) -> None:  #
noqa: ARG002
        self.register_match("visa-free-business-quick")

    @_register_rule(
        name="insurance-health-therapy",
        priority=276,
        recommendation=(
            "Оздоровительная поездка с медицинским блоком: страховка
делает формат безопаснее: "
            "например, Мертвое море, Карловы Вары, Кисловодск"
        ),
        conditions=(
            ConditionSpec(slot="insurance", op="eq", value="yes"),
            ConditionSpec(slot="travel_type", op="eq",
```

```
value="health"),
        ),
    )
    @Rule(TravelInput(insurance="yes", travel_type="health"))
    def rule_insurance_health_therapy(self) -> None:
        self.register_match("insurance-health-therapy")

    @_register_rule(
        name="service-premium-comfort",
        priority=275,
        recommendation=(
            "Комфортная поездка с премиальным сервисом и повышенным
уровнем удобства: "
            "например, Дубай, Сингапур, Майами"
        ),
        conditions=(
            ConditionSpec(slot="service_level", op="eq",
value="premium"),
            ConditionSpec(slot="budget_rub", op="gte", value=120000),
        ),
    )
    @Rule(
        TravelInput(service_level="premium",
budget_rub=MATCH.budget),
        TEST(lambda budget: budget >= 120000),
    )
    def rule_service_premium_comfort(self, budget: int) -> None:  #
noqa: ARG002
        self.register_match("service-premium-comfort")

    @_register_rule(
        name="service-economy-domestic",
        priority=274,
        recommendation=(
            "Экономичный маршрут внутри страны с упором на цену и
базовый набор услуг: "
            "например, частный сектор в Анапе, хостелы в Золотом
кольце"
        ),
        conditions=(
            ConditionSpec(slot="service_level", op="eq",
value="economy"),
            ConditionSpec(slot="budget_rub", op="lt", value=80000),
```

```
        ),
    )
    @Rule(
        TravelInput(service_level="economy",
budget_rub=MATCH.budget),
        TEST(lambda budget: budget < 80000),
    )
    def rule_service_economy_domestic(self, budget: int) -> None:  #
noqa: ARG002
        self.register_match("service-economy-domestic")

    @_register_rule(
        name="service-standard-culture",
        priority=273,
        recommendation=(
            "Культурный тур стандартного уровня: без излишней
роскоши, но с комфортным размещением: "
            "например, туры по Золотому кольцу, Псков, Рязань"
        ),
        conditions=(
            ConditionSpec(slot="service_level", op="eq",
value="standard"),
            ConditionSpec(slot="travel_type", op="eq",
value="culture"),
        ),
    )
    @Rule(TravelInput(service_level="standard",
travel_type="culture"))
    def rule_service_standard_culture(self) -> None:
        self.register_match("service-standard-culture")

    @_register_rule(
        name="hobby-hiking-eco",
        priority=272,
        recommendation=(
            "Природный маршрут с треккингом и умеренной физической
нагрузкой: "
            "например, экотропы Сочи, Алтай, Карелия"
        ),
        conditions=(
            ConditionSpec(slot="hobby", op="eq", value="hiking"),
            ConditionSpec(slot="travel_type", op="eq", value="eco"),
        ),
```

```
    )
    @Rule(TravelInput(hobby="hiking", travel_type="eco"))
    def rule_hobby_hiking_eco(self) -> None:
        self.register_match("hobby-hiking-eco")

    @_register_rule(
        name="hobby-food-gastro",
        priority=271,
        recommendation=(
            "Гастрономическая поездка с рынками, дегустациями и
локальной кухней: "
            "например, Италия, Франция, Грузия"
        ),
        conditions=(
            ConditionSpec(slot="hobby", op="eq", value="food"),
        ),
    )
    @Rule(TravelInput(hobby="food"))
    def rule_hobby_food_gastro(self) -> None:
        self.register_match("hobby-food-gastro")

    @_register_rule(
        name="hobby-museum-city",
        priority=270,
        recommendation=(
            "Городская экскурсионная поездка с музеями и
историческими центрами: "
            "например, Санкт-Петербург, Москва, Рим"
        ),
        conditions=(
            ConditionSpec(slot="hobby", op="eq", value="museum"),
        ),
    )
    @Rule(TravelInput(hobby="museum"))
    def rule_hobby_museum_city(self) -> None:
        self.register_match("hobby-museum-city")

    @_register_rule(
        name="family-relax-tour",
        priority=260,
        recommendation=(
            "Спокойный семейный отдых с понятной логистикой: "
            "например, Анапа, Суздаль, Карелия"
```

```
        ),
        conditions=(
            ConditionSpec(slot="companions", op="eq",
value="family"),
            ConditionSpec(slot="travel_type", op="eq",
value="relax"),
        ),
    )
    @Rule(TravelInput(companions="family", travel_type="relax"))
    def rule_family_relax_tour(self) -> None:
        self.register_match("family-relax-tour")

    @_register_rule(
        name="friends-active-tour",
        priority=259,
        recommendation=(
            "Активная поездка для друзей: спорт, прогулки и
насыщенная программа: "
            "например, Алтай, Карелия, Шерегеш"
        ),
        conditions=(
            ConditionSpec(slot="companions", op="eq",
value="friends"),
            ConditionSpec(slot="travel_type", op="eq",
value="active"),
        ),
    )
    @Rule(TravelInput(companions="friends", travel_type="active"))
    def rule_friends_active_tour(self) -> None:
        self.register_match("friends-active-tour")

    @_register_rule(
        name="business-general",
        priority=250,
        recommendation=(
            "Деловой формат поездки с проживанием в удобной локации:
"
            "например, Москва, Санкт-Петербург, Екатеринбург"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="business"),
        ),
```

```
    )
    @Rule(TravelInput(travel_type="business"))
    def rule_business_general(self) -> None:
        self.register_match("business-general")

    @_register_rule(
        name="eco-general",
        priority=249,
        recommendation=(
            "Экологический тур с природными локациями: "
            "например, Алтай, Карелия, Байкал"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq", value="eco"),
        ),
    )
    @Rule(TravelInput(travel_type="eco"))
    def rule_eco_general(self) -> None:
        self.register_match("eco-general")

    @_register_rule(
        name="education-general",
        priority=248,
        recommendation=(
            "Обучающий тур с курсами или мастер-классами: "
            "например, Италия (кулинария), Франция (язык), Индия
(йога)"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="education"),
        ),
    )
    @Rule(TravelInput(travel_type="education"))
    def rule_education_general(self) -> None:
        self.register_match("education-general")

    @_register_rule(
        name="health-general",
        priority=247,
        recommendation=(
            "Оздоровительный отдых с восстановительными процедурами:
"
```

```
            "например, Кисловодск, Мертвое море, Карловы Вары"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="health"),
        ),
    )
    @Rule(TravelInput(travel_type="health"))
    def rule_health_general(self) -> None:
        self.register_match("health-general")

    @_register_rule(
        name="culture-general",
        priority=246,
        recommendation=(
            "Культурно-познавательная поездка с экскурсиями: "
            "например, Золотое кольцо, Санкт-Петербург, Прага"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="culture"),
        ),
    )
    @Rule(TravelInput(travel_type="culture"))
    def rule_culture_general(self) -> None:
        self.register_match("culture-general")

    @_register_rule(
        name="relax-general",
        priority=245,
        recommendation=(
            "Спокойный отдых с комфортным размещением: "
            "например, Анапа, Геленджик, Крым"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="relax"),
        ),
    )
    @Rule(TravelInput(travel_type="relax"))
    def rule_relax_general(self) -> None:
        self.register_match("relax-general")
```

```
    @_register_rule(
        name="active-general",
        priority=244,
        recommendation=(
            "Активный формат поездки с походами или спортом: "
            "например, Алтай, Карелия, Хибины"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="active"),
        ),
    )
    @Rule(TravelInput(travel_type="active"))
    def rule_active_general(self) -> None:
        self.register_match("active-general")

    @_register_rule(
        name="mixed-general",
        priority=243,
        recommendation=(
            "Смешанный формат путешествия: баланс отдыха и экскурсий:
"
            "например, Сочи, Крым, Казань"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="mixed"),
        ),
    )
    @Rule(TravelInput(travel_type="mixed"))
    def rule_mixed_general(self) -> None:
        self.register_match("mixed-general")

    # ========== НОВЫЕ ПРАВИЛА (44-55) ==========

    @_register_rule(
        name="summer-active-family",
        priority=320,
        recommendation=(
            "Активный семейный отдых летом в тёплом климате:
аквапарки, батуты, прогулки: "
            "например, Сочи, Турция, Кипр"
        ),
```

```
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="warm"),
            ConditionSpec(slot="season", op="eq", value="summer"),
            ConditionSpec(slot="travel_type", op="eq",
value="active"),
            ConditionSpec(slot="companions", op="eq",
value="family"),
        ),
    )
    @Rule(
        TravelInput(climate="warm", season="summer",
travel_type="active", companions="family"),
    )
    def rule_summer_active_family(self) -> None:
        self.register_match("summer-active-family")

    @_register_rule(
        name="winter-culture-couple",
        priority=319,
        recommendation=(
            "Романтический зимний тур с музеями и уютными кафе: "
            "например, Санкт-Петербург, Прага, Вена"
        ),
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="cold"),
            ConditionSpec(slot="season", op="eq", value="winter"),
            ConditionSpec(slot="travel_type", op="eq",
value="culture"),
            ConditionSpec(slot="companions", op="eq",
value="couple"),
        ),
    )
    @Rule(
        TravelInput(climate="cold", season="winter",
travel_type="culture", companions="couple"),
    )
    def rule_winter_culture_couple(self) -> None:
        self.register_match("winter-culture-couple")

    @_register_rule(
        name="warm-eco-solo",
        priority=318,
        recommendation=(
```

```
            "Тёплый эко-маршрут для одного: трекинг, леса, вулканы: "
            "например, Канарские острова, Мадейра, Тенерифе"
        ),
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="warm"),
            ConditionSpec(slot="travel_type", op="eq", value="eco"),
            ConditionSpec(slot="companions", op="eq", value="solo"),
            ConditionSpec(slot="hobby", op="eq", value="hiking"),
        ),
    )
    @Rule(
        TravelInput(climate="warm", travel_type="eco",
companions="solo", hobby="hiking"),
    )
    def rule_warm_eco_solo(self) -> None:
        self.register_match("warm-eco-solo")

    @_register_rule(
        name="cold-adventure-friends",
        priority=317,
        recommendation=(
            "Зимнее приключение с друзьями с высоким бюджетом: "
            "например, Норвегия (фьорды), Исландия, Аляска"
        ),
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="cold"),
            ConditionSpec(slot="travel_type", op="eq",
value="active"),
            ConditionSpec(slot="budget_rub", op="gte", value=120000),
            ConditionSpec(slot="trip_days", op="gte", value=10),
            ConditionSpec(slot="companions", op="eq",
value="friends"),
            ConditionSpec(slot="insurance", op="eq", value="yes"),
        ),
    )
    @Rule(
        TravelInput(
            climate="cold",
            travel_type="active",
            budget_rub=MATCH.budget,
            trip_days=MATCH.days,
            companions="friends",
            insurance="yes",
```

```
        ),
        TEST(lambda budget, days: budget >= 120000 and days >= 10),
    )
    def rule_cold_adventure_friends(self, budget: int, days: int) ->
None:
        self.register_match("cold-adventure-friends")

    @_register_rule(
        name="relax-long-visa",
        priority=316,
        recommendation=(
            "Долгий расслабленный отпуск с визой и высоким сервисом:
"
            "например, Таиланд (Пхукет), Вьетнам (Нячанг),
Доминикана"
        ),
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="warm"),
            ConditionSpec(slot="travel_type", op="eq",
value="relax"),
            ConditionSpec(slot="budget_rub", op="gte", value=150000),
            ConditionSpec(slot="trip_days", op="gte", value=14),
            ConditionSpec(slot="service_level", op="eq",
value="premium"),
            ConditionSpec(slot="visa_mode", op="eq",
value="visa_ready"),
            ConditionSpec(slot="insurance", op="eq", value="yes"),
        ),
    )
    @Rule(
        TravelInput(
            climate="warm",
            travel_type="relax",
            budget_rub=MATCH.budget,
            trip_days=MATCH.days,
            service_level="premium",
            visa_mode="visa_ready",
            insurance="yes",
        ),
        TEST(lambda budget, days: budget >= 150000 and days >= 14),
    )
    def rule_relax_long_visa(self, budget: int, days: int) -> None:
        self.register_match("relax-long-visa")
```

```
    @_register_rule(
        name="culture-budget-solo",
        priority=315,
        recommendation=(
            "Бюджетный музейный тур для одного: недорогое жильё,
входные билеты: "
            "например, Псков, Смоленск, Рязань"
        ),
        conditions=(
            ConditionSpec(slot="travel_type", op="eq",
value="culture"),
            ConditionSpec(slot="budget_rub", op="gte", value=30000),
            ConditionSpec(slot="budget_rub", op="lt", value=60000),
            ConditionSpec(slot="trip_days", op="gte", value=4),
            ConditionSpec(slot="trip_days", op="lte", value=6),
            ConditionSpec(slot="companions", op="eq", value="solo"),
            ConditionSpec(slot="service_level", op="eq",
value="economy"),
            ConditionSpec(slot="hobby", op="eq", value="museum"),
        ),
    )
    @Rule(
        TravelInput(
            travel_type="culture",
            budget_rub=MATCH.budget,
            trip_days=MATCH.days,
            companions="solo",
            service_level="economy",
            hobby="museum",
        ),
        TEST(lambda budget, days: 30000 <= budget < 60000 and 4 <=
days <= 6),
    )
    def rule_culture_budget_solo(self, budget: int, days: int) ->
None:
        self.register_match("culture-budget-solo")

    @_register_rule(
        name="family-roadtrip-mild",
        priority=314,
        recommendation=(
            "Семейное автопутешествие по умеренному климату: "
```

```
            "например, Золотое кольцо, Карелия, Алтай на машине"
        ),
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="mild"),
            ConditionSpec(slot="season", op="eq", value="summer"),
            ConditionSpec(slot="travel_type", op="eq",
value="mixed"),
            ConditionSpec(slot="budget_rub", op="gte", value=50000),
            ConditionSpec(slot="budget_rub", op="lt", value=90000),
            ConditionSpec(slot="trip_days", op="gte", value=7),
            ConditionSpec(slot="trip_days", op="lte", value=9),
            ConditionSpec(slot="companions", op="eq",
value="family"),
            ConditionSpec(slot="service_level", op="eq",
value="economy"),
            ConditionSpec(slot="insurance", op="eq", value="no"),
        ),
    )
    @Rule(
        TravelInput(
            climate="mild",
            season="summer",
            travel_type="mixed",
            budget_rub=MATCH.budget,
            trip_days=MATCH.days,
            companions="family",
            service_level="economy",
            insurance="no",
        ),
        TEST(lambda budget, days: 50000 <= budget < 90000 and 7 <=
days <= 9),
    )
    def rule_family_roadtrip_mild(self, budget: int, days: int) ->
None:
        self.register_match("family-roadtrip-mild")

    @_register_rule(
        name="health-premium-spring",
        priority=313,
        recommendation=(
            "Весеннее оздоровление для пары с премиум-сервисом: "
            "например, Карловы Вары, Мертвое море, Италия (термы)"
        ),
```

```
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="mild"),
            ConditionSpec(slot="season", op="eq", value="spring"),
            ConditionSpec(slot="travel_type", op="eq",
value="health"),
            ConditionSpec(slot="budget_rub", op="gte", value=100000),
            ConditionSpec(slot="trip_days", op="gte", value=7),
            ConditionSpec(slot="trip_days", op="lte", value=10),
            ConditionSpec(slot="companions", op="eq",
value="couple"),
            ConditionSpec(slot="service_level", op="eq",
value="premium"),
            ConditionSpec(slot="insurance", op="eq", value="yes"),
        ),
    )
    @Rule(
        TravelInput(
            climate="mild",
            season="spring",
            travel_type="health",
            budget_rub=MATCH.budget,
            trip_days=MATCH.days,
            companions="couple",
            service_level="premium",
            insurance="yes",
        ),
        TEST(lambda budget, days: budget >= 100000 and 7 <= days <=
10),
    )
    def rule_health_premium_spring(self, budget: int, days: int) ->
None:
        self.register_match("health-premium-spring")

    @_register_rule(
        name="active-visa-free-short",
        priority=312,
        recommendation=(
            "Активный безвизовый тур с друзьями: трекинг, рафтинг,
горы: "
            "например, Армения, Грузия, Абхазия"
        ),
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="warm"),
```

```
            ConditionSpec(slot="travel_type", op="eq",
value="active"),
            ConditionSpec(slot="budget_rub", op="lt", value=80000),
            ConditionSpec(slot="trip_days", op="gte", value=4),
            ConditionSpec(slot="trip_days", op="lte", value=7),
            ConditionSpec(slot="companions", op="eq",
value="friends"),
            ConditionSpec(slot="service_level", op="eq",
value="economy"),
            ConditionSpec(slot="visa_mode", op="eq",
value="visa_free_only"),
            ConditionSpec(slot="insurance", op="eq", value="no"),
        ),
    )
    @Rule(
        TravelInput(
            climate="warm",
            travel_type="active",
            budget_rub=MATCH.budget,
            trip_days=MATCH.days,
            companions="friends",
            service_level="economy",
            visa_mode="visa_free_only",
            insurance="no",
        ),
        TEST(lambda budget, days: budget < 80000 and 4 <= days <= 7),
    )
    def rule_active_visa_free_short(self, budget: int, days: int) ->
None:
        self.register_match("active-visa-free-short")

    @_register_rule(
        name="business-long-economy",
        priority=311,
        recommendation=(
            "Длительная деловая поездка с минимальными расходами:
хостелы, коворкинги, трансферы: "
            "например, Минск, Казань, Екатеринбург"
        ),
        conditions=(
            ConditionSpec(slot="season", op="eq", value="autumn"),
            ConditionSpec(slot="travel_type", op="eq",
value="business"),
```

```
            ConditionSpec(slot="budget_rub", op="gte", value=50000),
            ConditionSpec(slot="budget_rub", op="lt", value=80000),
            ConditionSpec(slot="trip_days", op="gte", value=6),
            ConditionSpec(slot="trip_days", op="lte", value=10),
            ConditionSpec(slot="companions", op="eq", value="solo"),
            ConditionSpec(slot="service_level", op="eq",
value="economy"),
        ),
    )
    @Rule(
        TravelInput(
            season="autumn",
            travel_type="business",
            budget_rub=MATCH.budget,
            trip_days=MATCH.days,
            companions="solo",
            service_level="economy",
        ),
        TEST(lambda budget, days: 50000 <= budget < 80000 and 6 <=
days <= 10),
    )
    def rule_business_long_economy(self, budget: int, days: int) ->
None:
        self.register_match("business-long-economy")

    @_register_rule(
        name="mixed-family-culture",
        priority=310,
        recommendation=(
            "Семейный микс: половина дня экскурсии, половина — отдых:
"
            "например, Казань, Суздаль, Калининград"
        ),
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="mild"),
            ConditionSpec(slot="travel_type", op="eq",
value="mixed"),
            ConditionSpec(slot="trip_days", op="gte", value=5),
            ConditionSpec(slot="trip_days", op="lte", value=8),
            ConditionSpec(slot="companions", op="eq",
value="family"),
            ConditionSpec(slot="service_level", op="eq",
value="standard"),
```

```
            ConditionSpec(slot="hobby", op="eq", value="museum"),
        ),
    )
    @Rule(
        TravelInput(
            climate="mild",
            travel_type="mixed",
            trip_days=MATCH.days,
            companions="family",
            service_level="standard",
            hobby="museum",
        ),
        TEST(lambda days: 5 <= days <= 8),
    )
    def rule_mixed_family_culture(self, days: int) -> None:
        self.register_match("mixed-family-culture")

    @_register_rule(
        name="solo-eco-winter",
        priority=309,
        recommendation=(
            "Зимний эко-тур в одиночку с возможным гидом: "
            "например, Хибины, Байкал (лёд), Карелия"
        ),
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="cold"),
            ConditionSpec(slot="season", op="eq", value="winter"),
            ConditionSpec(slot="travel_type", op="eq", value="eco"),
            ConditionSpec(slot="budget_rub", op="gte", value=60000),
            ConditionSpec(slot="budget_rub", op="lt", value=100000),
            ConditionSpec(slot="trip_days", op="gte", value=5),
            ConditionSpec(slot="trip_days", op="lte", value=7),
            ConditionSpec(slot="companions", op="eq", value="solo"),
            ConditionSpec(slot="service_level", op="eq",
value="standard"),
            ConditionSpec(slot="hobby", op="eq", value="hiking"),
            ConditionSpec(slot="insurance", op="eq", value="yes"),
        ),
    )
    @Rule(
        TravelInput(
            climate="cold",
            season="winter",
```

```
            travel_type="eco",
            budget_rub=MATCH.budget,
            trip_days=MATCH.days,
            companions="solo",
            service_level="standard",
            hobby="hiking",
            insurance="yes",
        ),
        TEST(lambda budget, days: 60000 <= budget < 100000 and 5 <=
days <= 7),
    )
    def rule_solo_eco_winter(self, budget: int, days: int) -> None:
        self.register_match("solo-eco-winter")

    @Rule(TravelInput(), salience=-1000)
    def rule_default(self) -> None:
        if self.selected_rule is None:
            self.register_match(DEFAULT_RULE_NAME)


class TravelRuleEngine:
    """Rule engine based on experta with forward and backward explain
output."""

    def __init__(self) -> None:
        self.engine = _TravelExpertEngine()
        self.fact_types = {
            spec.fact_slot: spec.field_type
            for spec in TRAVEL_FACTS
            if spec.fact_slot is not None
        }

    def rules_count(self) -> int:
        return len(self.engine.rule_metadata) - 1

    def evaluate(
        self,
        facts: Mapping[str, Any],
        *,
        explain: bool = False,
    ) -> str | EvaluationResult:
        started = perf_counter()
```

```
        normalized = self._normalize_facts(facts)
        self.engine.reset_runtime_state()
        self.engine.reset()
        self.engine.declare(TravelInput(**normalized))
        self.engine.run()

        elapsed_ms = (perf_counter() - started) * 1000
        selected_rule = self.engine.selected_rule or
DEFAULT_RULE_NAME

        if explain:
            return EvaluationResult(
                recommendation=self.engine.recommendation,
                matched_rules=tuple(self.engine.matched_rules),
                selected_rule=selected_rule,
                elapsed_ms=round(elapsed_ms, 3),
                passes=3,
                steps=self._build_forward_steps(
                    normalized=normalized,
                    matched_rules=tuple(self.engine.matched_rules),
                    selected_rule=selected_rule,
                ),
            )

        return self.engine.recommendation

    def backward(
        self,
        *,
        goal: str,
        known_facts: Mapping[str, Any],
        explain: bool = True,
    ) -> bool | BackwardResult:
        started = perf_counter()
        normalized = self._normalize_facts(known_facts)
        metadata = self.engine.rule_metadata
        steps: list[dict[str, Any]] = []
        self._append_backward_step(steps, depth=0, step="prove-goal",
goal=goal)
        matched_rules: list[str] = []

        if goal == "*":
            candidates = _sorted_rules(metadata)
```

```
            self._append_backward_step(
                steps,
                depth=0,
                step="select-rules",
                goal=goal,
                candidates=[item.name for item in candidates],
            )
        else:
            candidate = metadata.get(goal)
            if candidate is None:
                self._append_backward_step(
                    steps,
                    depth=0,
                    step="goal-not-found",
                    goal=goal,
                )
                elapsed_ms = (perf_counter() - started) * 1000
                result = BackwardResult(
                    goal=goal,
                    achieved=False,
                    selected_rule=None,
                    matched_rules=(),
                    recommendation=None,
                    elapsed_ms=round(elapsed_ms, 3),
                    passes=max(step["pass"] for step in steps),
                    steps=tuple(steps),
                    proof={
                        "type": "goal",
                        "goal": goal,
                        "achieved": False,
                        "reason": "goal-not-found",
                    },
                )
                if explain:
                    return result
                return False
            candidates = [candidate]
            self._append_backward_step(
                steps,
                depth=0,
                step="select-rules",
                goal=goal,
                candidates=[candidate.name],
```

```
            )

        selected: RuleMetadata | None = None
        candidate_proofs: list[dict[str, Any]] = []

        for candidate in candidates:
            achieved_candidate, candidate_proof =
self._build_backward_rule_proof(
                candidate=candidate,
                known_facts=normalized,
                steps=steps,
                depth=1,
            )
            candidate_proofs.append(candidate_proof)
            if achieved_candidate:
                matched_rules.append(candidate.name)
                if selected is None:
                    selected = candidate
                    self._append_backward_step(
                        steps,
                        depth=0,
                        step="goal-proved",
                        goal=goal,
                        selected_rule=candidate.name,
                    )
                    if goal != "*":
                        break
                else:
                    self._append_backward_step(
                        steps,
                        depth=0,
                        step="candidate-also-matched",
                        goal=goal,
                        rule=candidate.name,
                        selected_rule=selected.name,
                    )

        achieved = selected is not None
        if selected is None and goal in {"*", DEFAULT_RULE_NAME}:
            selected = metadata[DEFAULT_RULE_NAME]
            achieved = True
            matched_rules.append(DEFAULT_RULE_NAME)
            self._append_backward_step(
```

```
                steps,
                depth=0,
                step="fallback-default",
                goal=goal,
                rule=DEFAULT_RULE_NAME,
            )

        if not achieved and goal not in {"*", DEFAULT_RULE_NAME}:
            self._append_backward_step(
                steps,
                depth=0,
                step="goal-failed",
                goal=goal,
            )

        elapsed_ms = (perf_counter() - started) * 1000
        result = BackwardResult(
            goal=goal,
            achieved=achieved,
            selected_rule=selected.name if selected else None,
            matched_rules=tuple(matched_rules),
            recommendation=selected.recommendation if selected else
None,
            elapsed_ms=round(elapsed_ms, 3),
            passes=max(step["pass"] for step in steps),
            steps=tuple(steps),
            proof={
                "type": "goal",
                "goal": goal,
                "achieved": achieved,
                "selected_rule": selected.name if selected else None,
                "matched_rules": list(matched_rules),
                "candidates": candidate_proofs,
            },
        )

        if explain:
            return result

        return achieved

    def _append_backward_step(
        self,
```

```
        steps: list[dict[str, Any]],
        *,
        depth: int,
        step: str,
        **payload: Any,
    ) -> None:
        steps.append(
            {
                "pass": depth + 1,
                "depth": depth,
                "step": step,
                **payload,
            }
        )

    def _build_backward_rule_proof(
        self,
        *,
        candidate: RuleMetadata,
        known_facts: Mapping[str, Any],
        steps: list[dict[str, Any]],
        depth: int,
    ) -> tuple[bool, dict[str, Any]]:
        self._append_backward_step(
            steps,
            depth=depth,
            step="try-rule",
            rule=candidate.name,
            priority=candidate.priority,
        )

        condition_proofs: list[dict[str, Any]] = []
        candidate_ok = True
        for condition in candidate.conditions:
            condition_ok, condition_proof =
self._build_backward_condition_proof(
                condition=condition,
                known_facts=known_facts,
                steps=steps,
                depth=depth + 1,
                rule_name=candidate.name,
            )
            condition_proofs.append(condition_proof)
```

```
            if not condition_ok:
                candidate_ok = False

        self._append_backward_step(
            steps,
            depth=depth,
            step="rule-proved" if candidate_ok else "rule-failed",
            rule=candidate.name,
        )
        return (
            candidate_ok,
            {
                "type": "rule",
                "rule": candidate.name,
                "priority": candidate.priority,
                "achieved": candidate_ok,
                "conditions": condition_proofs,
            },
        )

    def _build_backward_condition_proof(
        self,
        *,
        condition: ConditionSpec,
        known_facts: Mapping[str, Any],
        steps: list[dict[str, Any]],
        depth: int,
        rule_name: str,
    ) -> tuple[bool, dict[str, Any]]:
        self._append_backward_step(
            steps,
            depth=depth,
            step="prove-condition",
            rule=rule_name,
            slot=condition.slot,
            operator=condition.op,
            expected=condition.value,
        )

        actual = known_facts.get(condition.slot)
        if actual is None:
            self._append_backward_step(
                steps,
```

```
                depth=depth,
                step="condition-failed",
                rule=rule_name,
                slot=condition.slot,
                operator=condition.op,
                expected=condition.value,
                actual=None,
                reason="missing-fact",
            )
            return (
                False,
                {
                    "type": "condition",
                    "rule": rule_name,
                    "slot": condition.slot,
                    "operator": condition.op,
                    "expected": condition.value,
                    "actual": None,
                    "achieved": False,
                    "source": "facts",
                    "reason": "missing-fact",
                },
            )

        condition_ok = _apply_operator(condition.op, actual,
condition.value)
        self._append_backward_step(
            steps,
            depth=depth,
            step="condition-from-facts" if condition_ok else
"condition-failed",
            rule=rule_name,
            slot=condition.slot,
            operator=condition.op,
            expected=condition.value,
            actual=actual,
            matched=condition_ok,
        )
        return (
            condition_ok,
            {
                "type": "condition",
                "rule": rule_name,
```

```
                "slot": condition.slot,
                "operator": condition.op,
                "expected": condition.value,
                "actual": actual,
                "achieved": condition_ok,
                "source": "facts",
            },
        )

    def _build_forward_steps(
        self,
        *,
        normalized: Mapping[str, Any],
        matched_rules: tuple[str, ...],
        selected_rule: str,
    ) -> tuple[dict[str, Any], ...]:
        sorted_rules = _sorted_rules(self.engine.rule_metadata)
        steps: list[dict[str, Any]] = [
            {
                "pass": 1,
                "step": "declare-facts",
                "facts": dict(normalized),
            },
            {
                "pass": 2,
                "step": "select-candidates",
                "candidates": [item.name for item in sorted_rules],
            },
        ]
        matched_set = set(matched_rules)

        for candidate in sorted_rules:
            steps.append(
                {
                    "pass": 2,
                    "step": "inspect-rule",
                    "rule": candidate.name,
                    "priority": candidate.priority,
                }
            )
            candidate_ok = True
            for condition in candidate.conditions:
                condition_ok = _condition_is_satisfied(condition,
```

```
normalized)
                steps.append(
                    {
                        "pass": 2,
                        "step": "check-condition",
                        "rule": candidate.name,
                        "slot": condition.slot,
                        "operator": condition.op,
                        "expected": condition.value,
                        "actual": normalized.get(condition.slot),
                        "matched": condition_ok,
                    }
                )
                if not condition_ok:
                    candidate_ok = False

            if candidate_ok:
                steps.append(
                    {
                        "pass": 2,
                        "step": "rule-matched",
                        "rule": candidate.name,
                        "fired": candidate.name in matched_set,
                        "selected": candidate.name == selected_rule,
                    }
                )
            else:
                steps.append(
                    {
                        "pass": 2,
                        "step": "rule-rejected",
                        "rule": candidate.name,
                    }
                )

        steps.append(
            {
                "pass": 3,
                "step": "resolve-conflict",
                "matched_rules": list(matched_rules),
            }
        )
```

```
        if selected_rule == DEFAULT_RULE_NAME:
            steps.append(
                {
                    "pass": 3,
                    "step": "fallback-default",
                    "rule": DEFAULT_RULE_NAME,
                }
            )
        else:
            steps.append(
                {
                    "pass": 3,
                    "step": "select-rule",
                    "rule": selected_rule,
                    "matched_rules": list(matched_rules),
                }
            )

        return tuple(steps)

    def _normalize_facts(self, raw_facts: Mapping[str, Any]) ->
dict[str, Any]:
        normalized: dict[str, Any] = {}
        for slot, field_type in self.fact_types.items():
            value = raw_facts.get(slot)
            if value is None or value == "":
                continue
            if field_type == "integer":
                normalized[slot] = int(value)
            else:
                normalized[slot] = value
        return normalized


def init_rule_engine(app: "Flask") -> TravelRuleEngine:
    engine = TravelRuleEngine()
    app.extensions["expert_engine"] = engine
    return engine
```

Листинг А.3 — Добавление правил